

Physlib Monthly Report

1–31 July 2024

Contributors: Joseph Tooby-Smith, Pietro Monticone

Generated 14 August 2026 from commit 2439ec3.

Abstract

Physlib is a community library of formalised physics for the [Lean 4](#) proof assistant: definitions and results from across physics, stated in a formal language and checked by machine. This report is an automatically generated record of one month’s additions to it.

It covers 1–31 July 2024 on branch `master` of [leanprover-community/physlib](#), between commits `0116994` and `2439ec3`. Over that period 2 contributors landed 138 commits, changing 8,944 lines across 123 files and adding 398 new Lean declarations, each catalogued with its statement in Section 2. The complete diff is available at [GitHub compare](#).

Contents

1	Introduction	2
1.1	Contributors	2
1.2	Reviewers	2
1.3	Modified subfolders	2
1.4	New declarations by kind	2
2	Details by file	3
2.1	HepLean.AnomalyCancellation.MSSMNu.OrthogY3B3.PlaneWithY3B3	3
2.2	HepLean.AnomalyCancellation.MSSMNu.Permutations	3
2.3	HepLean.BeyondTheStandardModel.TwoHDM.Basic	3
2.4	HepLean.FeynmanDiagrams.Basic	4
2.5	HepLean.SpaceTime.LorentzAlgebra.Basic	4
2.6	HepLean.SpaceTime.LorentzGroup.Basic	4
2.7	HepLean.SpaceTime.LorentzGroup.Boosts	8
2.8	HepLean.SpaceTime.LorentzGroup.Orthochronous	9
2.9	HepLean.SpaceTime.LorentzGroup.Proper	11
2.10	HepLean.SpaceTime.LorentzGroup.Rotations	13
2.11	HepLean.SpaceTime.LorentzTensor.Basic	14
2.12	HepLean.SpaceTime.LorentzTensor.Contraction	21
2.13	HepLean.SpaceTime.LorentzTensor.Fin	25
2.14	HepLean.SpaceTime.LorentzTensor.MulActionTensor	26
2.15	HepLean.SpaceTime.LorentzTensor.Real.Basic	28
2.16	HepLean.SpaceTime.LorentzTensor.RisingLowering	28
2.17	HepLean.SpaceTime.LorentzVector.Basic	30
2.18	HepLean.SpaceTime.LorentzVector.Contraction	32
2.19	HepLean.SpaceTime.LorentzVector.Covariant	35
2.20	HepLean.SpaceTime.LorentzVector.LorentzAction	37
2.21	HepLean.SpaceTime.LorentzVector.NormOne	37

2.22	<code>HepLean.SpaceTime.MinkowskiMetric</code>	40
2.23	<code>HepLean.SpaceTime.SL2C.Basic</code>	44
2.24	<code>HepLean.StandardModel.HiggsBoson.Basic</code>	44
2.25	<code>HepLean.StandardModel.HiggsBoson.PointwiseInnerProd</code>	44
2.26	<code>HepLean.StandardModel.HiggsBoson.Potential</code>	45
2.27	<code>scripts.find_TODOs</code>	47
2.28	<code>scripts.hepLean_style_lint</code>	48
2.29	<code>scripts.mathlib_textLint_on_hepLean</code>	49
2.30	<code>scripts.openAI_doc_check</code>	49
2.31	<code>scripts.stats</code>	50

1 Introduction

During Jul 2024, 2 contributors submitted 138 commits that changed 8,944 lines (+6,023 / -2,921) across 123 files spanning 20 top-level areas of the repository. Of those, 26 files were newly added and 9 were removed outright. This report catalogues 398 newly added Lean declarations: 243 lemmas, 120 defs, 27 instances, 4 structures, 2 abbrevs, 1 class, and 1 inductive.

See the [full compare diff](#) and the [list of commits](#) on GitHub for the raw source.

1.1 Contributors

The following individuals authored commits merged during this reporting period, listed alphabetically.

Contributor	Lines Changed	Commits
Joseph Tooby-Smith	22,973	134
Pietro Monticone	127	4

1.2 Reviewers

The following individuals approved pull requests during this reporting period, listed alphabetically.

No reviewers identified for this month.

1.3 Modified subfolders

Subfolder	Files	New	Deleted	Additions	Deletions
HepLean/SpaceTime	28	+15	-3	+3,645	-1,168
HepLean/AnomalyCancellation	48	—	—	+653	-856
HepLean/StandardModel	7	+3	-1	+658	-450
scripts	12	+7	-2	+545	-15
HepLean/FlavorPhysics	7	—	—	+159	-207
HepLean/FeynmanDiagrams	4	—	—	+77	-103
HepLean/BeyondTheStandardModel	1	—	—	+86	-13
HepLean/Mathematics	2	—	—	+29	-43
docs	2	—	—	+44	-4
README.md	1	—	—	+24	-23
lake-manifest.json	1	—	—	+22	-12
lakefile.toml	1	—	—	+31	-3
.github/workflows	1	—	—	+16	-13
HepLean.lean	1	—	—	+19	-5
.vscode	1	+1	—	+10	-0
.lake	2	—	-2	+0	-4
.gitignore	1	—	—	+2	-1
docs/_layouts	1	—	—	+2	-0
lean-toolchain	1	—	—	+1	-1
.DS_Store	1	—	-1	+0	-0

1.4 New declarations by kind

Kind	Count
lemma	243
def	120
instance	27
structure	4
abbrev	2
class	1
inductive	1

2 Details by file

2.1 HepLean.AnomalyCancellation.MSSMNu.OrthogY3B3.PlaneWithY3B3

[HepLean/AnomalyCancellation/MSSMNu/OrthogY3B3/PlaneWithY3B3.lean](#) on GitHub.

def α_2 [source]

A helper function to simplify following expressions.

```
def  $\alpha_2$  (T : MSSMACC.AnomalyFreePerp) :  $\mathbb{Q}$ 
```

def α_3 [source]

A helper function to simplify following expressions.

```
def  $\alpha_3$  (T : MSSMACC.AnomalyFreePerp) :  $\mathbb{Q}$ 
```

lemma lineQuad_cube [source]

```
lemma lineQuad_cube (R : MSSMACC.AnomalyFreePerp) (c1 c2 c3 :  $\mathbb{Q}$ ) :
  accCube (lineQuad R c1 c2 c3).val =
    - 4 * (c1 * quadBiLin B3.val R.val - c2 * quadBiLin Y3.val R.val) ^ 2 *
      ( $\alpha_1$  R * c1 +  $\alpha_2$  R * c2 +  $\alpha_3$  R * c3)
```

2.2 HepLean.AnomalyCancellation.MSSMNu.Permutations

[HepLean/AnomalyCancellation/MSSMNu/Permutations.lean](#) on GitHub.

instance Group PermGroup [source]

```
@[simp]
instance : Group PermGroup
```

2.3 HepLean.BeyondTheStandardModel.TwoHDM.Basic

[HepLean/BeyondTheStandardModel/TwoHDM/Basic.lean](#) on GitHub.

lemma swap_fields [source]

Swapping ϕ_1 with ϕ_2 , and a number of the parameters (with possible conjugation) leads to an identical potential.

```

lemma swap_fields :
  potential m112 m222 ↕1 ↕2 ↕3 ↕4 m122 ↕5 ↕6 ↕7  $\Phi_1$   $\Phi_2$ 
= potential m222 m112 ↕2 ↕1 ↕3 ↕4 (conj m122) (conj ↕5) (conj ↕7) (conj ↕6)  $\Phi_2$   $\Phi_1$ 

```

lemma right_zero

[\[source\]](#)

If Φ_2 is zero the potential reduces to the Higgs potential on Φ_1 .

```

lemma right_zero : potential m112 m222 ↕1 ↕2 ↕3 ↕4 m122 ↕5 ↕6 ↕7  $\Phi_1$  0 =
  StandardModel.HiggsField.potential (- m112) (↕1/2)  $\Phi_1$ 

```

lemma left_zero

[\[source\]](#)

If Φ_1 is zero the potential reduces to the Higgs potential on Φ_2 .

```

lemma left_zero : potential m112 m222 ↕1 ↕2 ↕3 ↕4 m122 ↕5 ↕6 ↕7 0  $\Phi_2$  =
  StandardModel.HiggsField.potential (- m222) (↕2/2)  $\Phi_2$ 

```

def IsBounded

[\[source\]](#)

The proposition on the coefficients for a potential to be bounded.

```

def IsBounded (m112 m222 ↕1 ↕2 ↕3 ↕4 : ℝ) (m122 ↕5 ↕6 ↕7 : ℂ) : Prop

```

2.4 HepLean.FeynmanDiagrams.Basic

[HepLean/FeynmanDiagrams/Basic.lean](#) on GitHub.

instance Decidable (∀ x, G.⟨ $\mathcal{E}$ To $\mathcal{EV}$.hom (⟨ $\mathcal{E}$ x) = edgeVertexMap $\mathcal{E}$ $\mathcal{V}$ (F.⟨ $\mathcal{E}$...

[\[source\]](#)

```

instance {F G : FeynmanDiagram P} [IsFiniteDiagram F] [IsFiniteDiagram G] [
  IsFinitePreFeynmanRule P]
  (E : F.⟨ $\mathcal{E}$  → G.⟨ $\mathcal{E}$ ) (V : F.⟨ $\mathcal{V}$  → G.⟨ $\mathcal{V}$ ) (⟨E : F.⟨ $\mathcal{E}$  → G.⟨ $\mathcal{E}$ ) :
  Decidable (∀ x, G.⟨ $\mathcal{E}$ To $\mathcal{EV}$ .hom (⟨ $\mathcal{E}$  x) = edgeVertexMap  $\mathcal{E}$   $\mathcal{V}$  (F.⟨ $\mathcal{E}$ To $\mathcal{EV}$ .hom x))

```

2.5 HepLean.SpaceTime.LorentzAlgebra.Basic

[HepLean/SpaceTime/LorentzAlgebra/Basic.lean](#) on GitHub.

instance lorentzVectorAsLieRingModule

[\[source\]](#)

```

@[simps!]
instance lorentzVectorAsLieRingModule : LieRingModule lorentzAlgebra (LorentzVector 3) where

```

2.6 HepLean.SpaceTime.LorentzGroup.Basic

[HepLean/SpaceTime/LorentzGroup/Basic.lean](#) on GitHub.

def LorentzGroup

[\[source\]](#)

The Lorentz group is the subset of matrices which preserve the minkowski metric.

```

def LorentzGroup (d : ℕ) : Set (Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ)

```

lemma mem_iff_norm[\[source\]](#)

```
lemma mem_iff_norm :  $\Lambda \in \text{LorentzGroup } d \leftrightarrow$   

 $\forall (x : \text{LorentzVector } d), \langle \Lambda *_v x, \Lambda *_v x \rangle_m = \langle x, x \rangle_m$ 
```

lemma mem_iff_on_right[\[source\]](#)

```
lemma mem_iff_on_right :  $\Lambda \in \text{LorentzGroup } d \leftrightarrow$   

 $\forall (x y : \text{LorentzVector } d), \langle x, (\text{dual } \Lambda * \Lambda) *_v y \rangle_m = \langle x, y \rangle_m$ 
```

lemma mem_iff_dual_mul_self[\[source\]](#)

```
lemma mem_iff_dual_mul_self :  $\Lambda \in \text{LorentzGroup } d \leftrightarrow \text{dual } \Lambda * \Lambda = 1$ 
```

lemma mem_iff_self_mul_dual[\[source\]](#)

```
lemma mem_iff_self_mul_dual :  $\Lambda \in \text{LorentzGroup } d \leftrightarrow \Lambda * \text{dual } \Lambda = 1$ 
```

lemma mem_iff_transpose[\[source\]](#)

```
lemma mem_iff_transpose :  $\Lambda \in \text{LorentzGroup } d \leftrightarrow \Lambda^T \in \text{LorentzGroup } d$ 
```

lemma mem_mul[\[source\]](#)

```
lemma mem_mul (h $\Lambda$  :  $\Lambda \in \text{LorentzGroup } d$ ) (h $\Lambda'$  :  $\Lambda' \in \text{LorentzGroup } d$ ) :  $\Lambda * \Lambda' \in \text{LorentzGroup } d$ 
```

lemma one_mem[\[source\]](#)

```
lemma one_mem :  $1 \in \text{LorentzGroup } d$ 
```

lemma dual_mem[\[source\]](#)

```
lemma dual_mem (h :  $\Lambda \in \text{LorentzGroup } d$ ) :  $\text{dual } \Lambda \in \text{LorentzGroup } d$ 
```

instance lorentzGroupIsGroup[\[source\]](#)

```
@[simp mul_coe one_coe inv div]  
instance lorentzGroupIsGroup : Group (LorentzGroup d) where
```

instance TopologicalSpace (LorentzGroup d)[\[source\]](#)

LorentzGroup has the subtype topology.

```
instance : TopologicalSpace (LorentzGroup d)
```

lemma coe_inv[\[source\]](#)

```
lemma coe_inv :  $(\Lambda^{-1}).1 = \Lambda.1^{-1}$ 
```

lemma subtype_inv_mul[\[source\]](#)

@[simp]

lemma subtype_inv_mul : (Subtype.val $\wedge$)⁻¹ * (Subtype.val $\wedge$) = 1**lemma subtype_mul_inv**[\[source\]](#)

@[simp]

lemma subtype_mul_inv : (Subtype.val $\wedge$) * (Subtype.val $\wedge$)⁻¹ = 1**lemma mul_minkowskiMatrix_mul_transpose**[\[source\]](#)

@[simp]

lemma mul_minkowskiMatrix_mul_transpose :
(Subtype.val $\wedge$) * minkowskiMatrix * (Subtype.val $\wedge$).transpose = minkowskiMatrix**lemma transpose_mul_minkowskiMatrix_mul_self**[\[source\]](#)

@[simp]

lemma transpose_mul_minkowskiMatrix_mul_self :
(Subtype.val $\wedge$).transpose * minkowskiMatrix * (Subtype.val $\wedge$) = minkowskiMatrix**def transpose**[\[source\]](#)*The transpose of a matrix in the Lorentz group is an element of the Lorentz group.***def** transpose ($\wedge$: LorentzGroup d) : LorentzGroup d**lemma transpose_one**[\[source\]](#)

@[simp]

lemma transpose_one : @transpose d 1 = 1**lemma transpose_mul**[\[source\]](#)

@[simp]

lemma transpose_mul : transpose ($\wedge$ * $\wedge'$) = transpose $\wedge'$ * transpose $\wedge$ **def toGL**[\[source\]](#)*The homomorphism of the Lorentz group into GL (Fin 4) $\mathbb{R}$.***def** toGL : LorentzGroup d →* GL (Fin 1 ⊕ Fin d) $\mathbb{R}$ **where****lemma toGL_injective**[\[source\]](#)**lemma** toGL_injective : Function.Injective (@toGL d)**def toProd**[\[source\]](#)*The homomorphism from the Lorentz Group into the monoid of matrices times the opposite of the monoid of matrices.*

```
@[simps!]
def toProd : LorentzGroup d →* (Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ) ×
  (Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ)m ∘ P
```

lemma toProd_eq_transpose_η

[\[source\]](#)

```
lemma toProd_eq_transpose_η : toProd Λ = (Λ.1, MulOpposite.op $ minkowskiMetric.dual Λ.1)
```

lemma toProd_injective

[\[source\]](#)

```
lemma toProd_injective : Function.Injective (@toProd d)
```

lemma toProd_continuous

[\[source\]](#)

```
lemma toProd_continuous : Continuous (@toProd d)
```

lemma toProd_embedding

[\[source\]](#)

The embedding from the Lorentz Group into the monoid of matrices times the opposite of the monoid of matrices.

```
lemma toProd_embedding : Embedding (@toProd d) where
```

lemma toGL_embedding

[\[source\]](#)

The embedding from the Lorentz Group into GL (Fin 4) ℝ.

```
lemma toGL_embedding : Embedding (@toGL d).toFun where
```

instance TopologicalGroup (LorentzGroup d)

[\[source\]](#)

```
instance : TopologicalGroup (LorentzGroup d)
```

def toNormOneLorentzVector

[\[source\]](#)

The first column of a Lorentz matrix as a NormOneLorentzVector.

```
@[simps!]
def toNormOneLorentzVector (Λ : LorentzGroup d) : NormOneLorentzVector d
```

def timeComp

[\[source\]](#)

The time like element of a Lorentz matrix.

```
@[simp]
def timeComp (Λ : LorentzGroup d) : ℝ
```

lemma timeComp_eq_toNormOneLorentzVector

[\[source\]](#)

```
lemma timeComp_eq_toNormOneLorentzVector : timeComp Λ = (toNormOneLorentzVector Λ).1.time
```


lemma timeComp_mul[\[source\]](#)

```
lemma timeComp_mul (Λ Λ' : LorentzGroup d) : timeComp (Λ * Λ') =
  ⟨⟨toNormOneLorentzVector (transpose Λ), (toNormOneLorentzVector Λ').1.spaceReflection⟩⟩m
```

2.7 HepLean.SpaceTime.LorentzGroup.Boosts

[HepLean/SpaceTime/LorentzGroup/Boosts.lean](#) on GitHub.

def genBoostAux₁[\[source\]](#)

An auxillary linear map used in the definition of a generalised boost.

```
def genBoostAux1 (u v : FuturePointing d) : LorentzVector d →l[ℝ] LorentzVector d where
```

def genBoostAux₂[\[source\]](#)

An auxillary linear map used in the definition of a generalised boost.

```
def genBoostAux2 (u v : FuturePointing d) : LorentzVector d →l[ℝ] LorentzVector d where
```

def genBoost[\[source\]](#)

An generalised boost. This is a Lorentz transformation which takes the four velocity u to v .

```
def genBoost (u v : FuturePointing d) : LorentzVector d →l[ℝ] LorentzVector d
```

lemma self[\[source\]](#)

This lemma states that for a given four-velocity u , the general boost transformation $genBoost\ u\ u$ is equal to the identity linear map $LinearMap.id$.

```
lemma self (u : FuturePointing d) : genBoost u u = LinearMap.id
```

def toMatrix[\[source\]](#)

A generalised boost as a matrix.

```
def toMatrix (u v : FuturePointing d) : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ
```

lemma toMatrix_mulVec[\[source\]](#)

```
lemma toMatrix_mulVec (u v : FuturePointing d) (x : LorentzVector d) :
  (toMatrix u v).mulVec x = genBoost u v x
```

lemma toMatrix_apply[\[source\]](#)

`@[simp]`

```
lemma toMatrix_apply (u v : FuturePointing d) (μ v : Fin 1 ⊕ Fin d) :
  (toMatrix u v) μ v = η μ μ * (⟨⟨e μ, e v⟩⟩m + 2 * ⟨⟨e v, u⟩⟩m * ⟨⟨e μ, v⟩⟩m
  - ⟨⟨e μ, u + v⟩⟩m * ⟨⟨e v, u + v⟩⟩m / (1 + ⟨⟨u, v.1.1⟩⟩m))
```

lemma toMatrix_continuous[\[source\]](#)

```
lemma toMatrix_continuous (u : FuturePointing d) : Continuous (toMatrix u)
```

lemma toMatrix_in_lorentzGroup[\[source\]](#)

```
lemma toMatrix_in_lorentzGroup (u v : FuturePointing d) : (toMatrix u v) ∈ LorentzGroup d
```

def toLorentz[\[source\]](#)

A generalised boost as an element of the Lorentz Group.

```
def toLorentz (u v : FuturePointing d) : LorentzGroup d
```

lemma toLorentz_continuous[\[source\]](#)

```
lemma toLorentz_continuous (u : FuturePointing d) : Continuous (toLorentz u)
```

lemma toLorentz_joined_to_1[\[source\]](#)

```
lemma toLorentz_joined_to_1 (u v : FuturePointing d) : Joined 1 (toLorentz u v)
```

lemma toLorentz_in_connected_component_1[\[source\]](#)

```
lemma toLorentz_in_connected_component_1 (u v : FuturePointing d) :  
  toLorentz u v ∈ connectedComponent 1
```

lemma isProper[\[source\]](#)

```
lemma isProper (u v : FuturePointing d) : IsProper (toLorentz u v)
```

2.8 HepLean.SpaceTime.LorentzGroup.Orthochronous

[HepLean/SpaceTime/LorentzGroup/Orthochronous.lean](#) on GitHub.

def IsOrthochronous[\[source\]](#)

A Lorentz transformation is orthochronous if its θ element is non-negative.

```
def IsOrthochronous : Prop
```

lemma IsOrthochronous_iff_futurePointing[\[source\]](#)

```
lemma IsOrthochronous_iff_futurePointing :  
  IsOrthochronous  $\Lambda \leftrightarrow$  (toNormOneLorentzVector  $\Lambda$ ) ∈ NormOneLorentzVector.FuturePointing d
```

lemma IsOrthochronous_iff_transpose[\[source\]](#)

```
lemma IsOrthochronous_iff_transpose :  
  IsOrthochronous  $\Lambda \leftrightarrow$  IsOrthochronous (transpose  $\Lambda$ )
```

lemma IsOrthochronous_iff_ge_one[\[source\]](#)

```
lemma IsOrthochronous_iff_ge_one :
  IsOrthochronous  $\Lambda \leftrightarrow 1 \leq \text{timeComp } \Lambda$ 
```

lemma not_orthochronous_iff_le_neg_one[\[source\]](#)

```
lemma not_orthochronous_iff_le_neg_one :
   $\neg \text{IsOrthochronous } \Lambda \leftrightarrow \text{timeComp } \Lambda \leq -1$ 
```

lemma not_orthochronous_iff_le_zero[\[source\]](#)

```
lemma not_orthochronous_iff_le_zero :
   $\neg \text{IsOrthochronous } \Lambda \leftrightarrow \text{timeComp } \Lambda \leq 0$ 
```

def timeCompCont[\[source\]](#)

The continuous map taking a Lorentz transformation to its 0 0 element.

```
def timeCompCont : C(LorentzGroup d,  $\mathbb{R}$ )
```

def stepFunction[\[source\]](#)

An auxillary function used in the definition of orthchroMapReal.

```
def stepFunction :  $\mathbb{R} \rightarrow \mathbb{R}$ 
```

lemma stepFunction_continuous[\[source\]](#)

```
lemma stepFunction_continuous : Continuous stepFunction
```

def orthchroMapReal[\[source\]](#)

The continuous map from LorentzGroup to $\mathbb{R}$ wh taking Orthochronous elements to 1 and non-orthochronous to -1.

```
def orthchroMapReal : C(LorentzGroup d,  $\mathbb{R}$ )
```

lemma orthchroMapReal_on_IsOrthochronous[\[source\]](#)

```
lemma orthchroMapReal_on_IsOrthochronous { $\Lambda$  : LorentzGroup d} (h : IsOrthochronous  $\Lambda$ ) :
  orthchroMapReal  $\Lambda$  = 1
```

lemma orthchroMapReal_on_not_IsOrthochronous[\[source\]](#)

```
lemma orthchroMapReal_on_not_IsOrthochronous { $\Lambda$  : LorentzGroup d} (h :  $\neg \text{IsOrthochronous } \Lambda$ ) :
  orthchroMapReal  $\Lambda$  = - 1
```

lemma orthchroMapReal_minus_one_or_one[\[source\]](#)

```
lemma orthchroMapReal_minus_one_or_one ( $\Lambda$  : LorentzGroup d) :
  orthchroMapReal  $\Lambda$  = -1  $\vee$  orthchroMapReal  $\Lambda$  = 1
```

def orthchroMap[\[source\]](#)

A continuous map from LorentzGroup to $\mathbb{Z}_2$ whose kernel are the Orthochronous elements.

```
def orthchroMap : C(LorentzGroup d,  $\mathbb{Z}_2$ )
```

lemma orthchroMap_IsOrthochronous[\[source\]](#)

```
lemma orthchroMap_IsOrthochronous { $\Lambda$  : LorentzGroup d} (h : IsOrthochronous  $\Lambda$ ) :  
  orthchroMap  $\Lambda$  = 1
```

lemma orthchroMap_not_IsOrthochronous[\[source\]](#)

```
lemma orthchroMap_not_IsOrthochronous { $\Lambda$  : LorentzGroup d} (h :  $\neg$  IsOrthochronous  $\Lambda$ ) :  
  orthchroMap  $\Lambda$  = Additive.toMul (1 : ZMod 2)
```

lemma mul_othchron_of_othchron_othchron[\[source\]](#)

```
lemma mul_othchron_of_othchron_othchron { $\Lambda$   $\Lambda'$  : LorentzGroup d} (h : IsOrthochronous  $\Lambda$ )  
  (h' : IsOrthochronous  $\Lambda'$ ) : IsOrthochronous ( $\Lambda * \Lambda'$ )
```

lemma mul_othchron_of_not_othchron_not_othchron[\[source\]](#)

```
lemma mul_othchron_of_not_othchron_not_othchron { $\Lambda$   $\Lambda'$  : LorentzGroup d} (h :  $\neg$  IsOrthochronous  
   $\Lambda$ )  
  (h' :  $\neg$  IsOrthochronous  $\Lambda'$ ) : IsOrthochronous ( $\Lambda * \Lambda'$ )
```

lemma mul_not_othchron_of_othchron_not_othchron[\[source\]](#)

```
lemma mul_not_othchron_of_othchron_not_othchron { $\Lambda$   $\Lambda'$  : LorentzGroup d} (h : IsOrthochronous  $\Lambda$ )  
  (h' :  $\neg$  IsOrthochronous  $\Lambda'$ ) :  $\neg$  IsOrthochronous ( $\Lambda * \Lambda'$ )
```

lemma mul_not_othchron_of_not_othchron_othchron[\[source\]](#)

```
lemma mul_not_othchron_of_not_othchron_othchron { $\Lambda$   $\Lambda'$  : LorentzGroup d} (h :  $\neg$  IsOrthochronous  
   $\Lambda$ )  
  (h' : IsOrthochronous  $\Lambda'$ ) :  $\neg$  IsOrthochronous ( $\Lambda * \Lambda'$ )
```

def orthchroRep[\[source\]](#)

The homomorphism from LorentzGroup to $\mathbb{Z}_2$ whose kernel are the Orthochronous elements.

```
def orthchroRep : LorentzGroup d  $\rightarrow^*$   $\mathbb{Z}_2$  where
```

2.9 HepLean.SpaceTime.LorentzGroup.Proper

[HepLean/SpaceTime/LorentzGroup/Proper.lean](#) on GitHub.

lemma det_eq_one_or_neg_one

[\[source\]](#)

The determinant of a member of the Lorentz group is 1 or -1.

```
lemma det_eq_one_or_neg_one (Λ : ℒ d) : Λ.1.det = 1 ∨ Λ.1.det = -1
```

instance TopologicalSpace ℤ₂

[\[source\]](#)

```
instance : TopologicalSpace ℤ2
```

instance DiscreteTopology ℤ₂

[\[source\]](#)

```
instance : DiscreteTopology ℤ2
```

instance TopologicalGroup ℤ₂

[\[source\]](#)

```
instance : TopologicalGroup ℤ2
```

def coeForℤ₂

[\[source\]](#)

A continuous function from $(\{-1, 1\} : \text{Set } \mathbb{R})$ to $\mathbb{Z}_2$.

```
@[simps!]
def coeForℤ2 : C(({-1, 1} : Set ℝ), ℤ2) where
```

def detContinuous

[\[source\]](#)

The continuous map taking a Lorentz matrix to its determinant.

```
def detContinuous : C(ℒ d, ℤ2)
```

lemma detContinuous_eq_iff_det_eq

[\[source\]](#)

```
lemma detContinuous_eq_iff_det_eq (Λ Λ' : LorentzGroup d) :
  detContinuous Λ = detContinuous Λ' ↔ Λ.1.det = Λ'.1.det
```

def detRep

[\[source\]](#)

The representation taking a Lorentz matrix to its determinant.

```
@[simps!]
def detRep : ℒ d →* ℤ2 where
```

lemma detRep_continuous

[\[source\]](#)

```
lemma detRep_continuous : Continuous (@detRep d)
```

lemma det_on_connected_component

[\[source\]](#)

```
lemma det_on_connected_component {Λ Λ' : LorentzGroup d} (h : Λ' ∈ connectedComponent Λ) :
  Λ.1.det = Λ'.1.det
```

lemma detRep_on_connected_component[\[source\]](#)

```
lemma detRep_on_connected_component {Λ Λ' : LorentzGroup d} (h : Λ' ∈ connectedComponent Λ) :
  detRep Λ = detRep Λ'
```

lemma det_of_joined[\[source\]](#)

```
lemma det_of_joined {Λ Λ' : LorentzGroup d} (h : Joined Λ Λ') : Λ.1.det = Λ'.1.det
```

def IsProper[\[source\]](#)

A Lorentz Matrix is proper if its determinant is 1.

```
@[simp]
```

```
def IsProper (Λ : LorentzGroup d) : Prop
```

instance DecidablePred (@IsProper d)[\[source\]](#)

```
instance : DecidablePred (@IsProper d)
```

lemma IsProper_iff[\[source\]](#)

```
lemma IsProper_iff (Λ : LorentzGroup d) : IsProper Λ ↔ detRep Λ = 1
```

lemma id_IsProper[\[source\]](#)

```
lemma id_IsProper : @IsProper d 1
```

lemma isProper_on_connected_component[\[source\]](#)

```
lemma isProper_on_connected_component {Λ Λ' : LorentzGroup d} (h : Λ' ∈ connectedComponent Λ) :
  IsProper Λ ↔ IsProper Λ'
```

2.10 HepLean.SpaceTime.LorentzGroup.Rotations

[HepLean/SpaceTime/LorentzGroup/Rotations.lean](#) on GitHub.

def S03ToMatrix[\[source\]](#)

Given a element of $S0(3)$ the matrix corresponding to a space-time rotation.

```
@[simp]
```

```
def S03ToMatrix (A : S0(3)) : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ
```

lemma S03ToMatrix_in_LorentzGroup[\[source\]](#)

```
lemma S03ToMatrix_in_LorentzGroup (A : S0(3)) : S03ToMatrix A ∈ LorentzGroup 3
```

lemma S03ToMatrix_injective[\[source\]](#)

```
lemma S03ToMatrix_injective : Function.Injective S03ToMatrix
```

def S03ToLorentz[\[source\]](#)

Given a element of $SO(3)$ the element of the Lorentz group corresponding to a space-time rotation.

```
def S03ToLorentz : S0(3) →* LorentzGroup 3 where
```

2.11 HepLean.SpaceTime.LorentzTensor.Basic

[HepLean/SpaceTime/LorentzTensor/Basic.lean](#) on GitHub.

def contrLeftAux[\[source\]](#)

An auxillary function to contract the vector space $V1$ and $V2$ in $V1 \otimes [R] V2 \otimes [R] V3$.

```
def contrLeftAux {V1 V2 V3 : Type} [AddCommMonoid V1] [AddCommMonoid V2] [AddCommMonoid V3]
[Module R V1] [Module R V2] [Module R V3] (f : V1 ⊗[R] V2 →l[R] R) :
V1 ⊗[R] V2 ⊗[R] V3 →l[R] V3
```

def contrRightAux[\[source\]](#)

An auxillary function to contract the vector space $V1$ and $V2$ in $(V3 \otimes [R] V1) \otimes [R] V2$.

```
def contrRightAux {V1 V2 V3 : Type} [AddCommMonoid V1] [AddCommMonoid V2] [AddCommMonoid V3]
[Module R V1] [Module R V2] [Module R V3] (f : V1 ⊗[R] V2 →l[R] R) :
(V3 ⊗[R] V1) ⊗[R] V2 →l[R] V3
```

def contrMidAux[\[source\]](#)

An auxillary function to contract the vector space $V1$ and $V2$ in $V4 \otimes [R] V1 \otimes [R] V2 \otimes [R] V3$.

```
def contrMidAux {V1 V2 V3 V4 : Type} [AddCommMonoid V1] [AddCommMonoid V2] [AddCommMonoid V3]
[AddCommMonoid V4] [Module R V1] [Module R V2] [Module R V3] [Module R V4]
(f : V1 ⊗[R] V2 →l[R] R) : (V4 ⊗[R] V1) ⊗[R] (V2 ⊗[R] V3) →l[R] V4 ⊗[R] V3
```

lemma contrRightAux_comp[\[source\]](#)

```
lemma contrRightAux_comp {V1 V2 V3 V4 V5 : Type} [AddCommMonoid V1] [AddCommMonoid V2]
[AddCommMonoid V3] [AddCommMonoid V4] [AddCommMonoid V5] [Module R V1] [Module R V3]
[Module R V2] [Module R V4] [Module R V5] (f : V2 ⊗[R] V3 →l[R] R) (g : V4 ⊗[R] V5 →l[R] R)
:
(contrRightAux f ∘l TensorProduct.map (LinearMap.id : V1 ⊗[R] V2 →l[R] V1 ⊗[R] V2)
(contrRightAux g)) =
(contrRightAux g) ∘l TensorProduct.map (contrMidAux f) LinearMap.id
∘l (TensorProduct.assoc R _ _).symm.toLinearMap
```

structure TensorStructure[\[source\]](#)

An initial structure specifying a tensor system (e.g. a system in which you can define real Lorentz tensors or Einstein notation convention).

```
structure TensorStructure (R : Type) [CommSemiring R] where
Color : Type
```

```

ColorModule : Color → Type
τ : Color → Color
τ_involutive : Function.Involutive τ
colorModule_addCommMonoid : ∀ μ, AddCommMonoid (ColorModule μ)
colorModule_module : ∀ μ, Module R (ColorModule μ)
contrDual : ∀ μ, ColorModule μ ⊗[R] ColorModule (τ μ) →l[R] R
contrDual_symm : ∀ μ × y, (contrDual μ) (x ⊗t[R] y) =
  (contrDual (τ μ)) (y ⊗t[R] (Equiv.cast (congrArg ColorModule (τ_involutive μ).symm) x))
unit : (μ : Color) → ColorModule (τ μ) ⊗[R] ColorModule μ
unit_rid : ∀ μ (x : ColorModule μ),
  TensorStructure.contrLeftAux (contrDual μ) (x ⊗t[R] unit μ) = x
metric : (μ : Color) → ColorModule μ ⊗[R] ColorModule μ
metric_dual : ∀ (μ : Color), (TensorStructure.contrMidAux (contrDual μ)
  (metric μ ⊗t[R] metric (τ μ))) = TensorProduct.comm _ _ _ (unit μ)

```

instance AddCommMonoid ($\mathcal{T}$.ColorModule μ)

[\[source\]](#)

instance : AddCommMonoid ($\mathcal{T}$.ColorModule μ)

instance Module R ($\mathcal{T}$.ColorModule μ)

[\[source\]](#)

instance : Module R ($\mathcal{T}$.ColorModule μ)

def Tensor

[\[source\]](#)

The type of tensors given a map from an indexing set X to the type of colors, specifying the color of that index.

def Tensor (c : $X \rightarrow \mathcal{T}$.Color) : Type

instance AddCommMonoid ($\mathcal{T}$.Tensor cX)

[\[source\]](#)

instance : AddCommMonoid ($\mathcal{T}$.Tensor cX)

instance Module R ($\mathcal{T}$.Tensor cX)

[\[source\]](#)

instance : Module R ($\mathcal{T}$.Tensor cX)

def colorModuleCast

[\[source\]](#)

Equivalence of ColorModule given an equality of colors.

def colorModuleCast (h : $\mu = \nu$) : $\mathcal{T}$.ColorModule $\mu \approx_l[R] \mathcal{T}$.ColorModule ν **where**

def colorRel

[\[source\]](#)

A relation on colors which is true if the two colors are equal or are duals.

def colorRel ($\mu \nu : \mathcal{T}$.Color) : Prop

lemma colorRel_equivalence[\[source\]](#)

An equivalence relation on colors which is true if the two colors are equal or are duals.

```
lemma colorRel_equivalence : Equivalence  $\mathcal{T}$ .colorRel where
```

instance colorSetoid[\[source\]](#)

The structure of a setoid on colors, two colors are related if they are equal, or dual.

```
instance colorSetoid : Setoid  $\mathcal{T}$ .Color
```

def colorQuot[\[source\]](#)

A map taking a color to its equivalence class in colorSetoid.

```
def colorQuot ( $\mu$  :  $\mathcal{T}$ .Color) : Quotient  $\mathcal{T}$ .colorSetoid
```

lemma tensorProd_piTensorProd_ext[\[source\]](#)

```
lemma tensorProd_piTensorProd_ext {M : Type} [AddCommMonoid M] [Module R M]
  {f g :  $\mathcal{T}$ .Tensor cX  $\otimes$ [R]  $\mathcal{T}$ .Tensor cY  $\rightarrow_l$ [R] M}
  (h :  $\forall$  p q, f (PiTensorProduct.tprod R p  $\otimes_t$ [R] PiTensorProduct.tprod R q)
    = g (PiTensorProduct.tprod R p  $\otimes_t$ [R] PiTensorProduct.tprod R q)) : f = g
```

def mapIso[\[source\]](#)

An linear equivalence of tensor spaces given a color-preserving equivalence of indexing sets.

```
def mapIso {c : X  $\rightarrow$   $\mathcal{T}$ .Color} {d : Y  $\rightarrow$   $\mathcal{T}$ .Color} (e : X  $\approx$  Y) (h : c = d  $\circ$  e) :
   $\mathcal{T}$ .Tensor c  $\approx_l$ [R]  $\mathcal{T}$ .Tensor d
```

lemma mapIso_trans_cond[\[source\]](#)

```
lemma mapIso_trans_cond {e : X  $\approx$  Y} {e' : Y  $\approx$  Z} (h : cX = cY  $\circ$  e) (h' : cY = cZ  $\circ$  e') :
  cX = cZ  $\circ$  (e.trans e')
```

lemma mapIso_trans[\[source\]](#)

```
@[simp]
lemma mapIso_trans (e : X  $\approx$  Y) (e' : Y  $\approx$  Z) (h : cX = cY  $\circ$  e) (h' : cY = cZ  $\circ$  e') :
  ( $\mathcal{T}$ .mapIso e h  $\lll$  $_l$   $\mathcal{T}$ .mapIso e' h') =  $\mathcal{T}$ .mapIso (e.trans e') ( $\mathcal{T}$ .mapIso_trans_cond h h')
```

lemma mapIso_mapIso[\[source\]](#)

```
@[simp]
lemma mapIso_mapIso (e : X  $\approx$  Y) (e' : Y  $\approx$  Z) (h : cX = cY  $\circ$  e) (h' : cY = cZ  $\circ$  e')
  (T :  $\mathcal{T}$ .Tensor cX) :
  ( $\mathcal{T}$ .mapIso e' h') ( $\mathcal{T}$ .mapIso e h T) =  $\mathcal{T}$ .mapIso (e.trans e') ( $\mathcal{T}$ .mapIso_trans_cond h h') T
```

lemma mapIso_symm[\[source\]](#)

```
@[simp]
lemma mapIso_symm (e : X ≈ Y) (h : cX = cY ∘ e) :
  (T.mapIso e h).symm = T.mapIso e.symm ((Equiv.eq_comp_symm e cY cX).mpr h.symm)
```

lemma mapIso_refl[\[source\]](#)

```
@[simp]
lemma mapIso_refl : T.mapIso (Equiv.refl X) (rfl : cX = cX) = LinearEquiv.refl R _
```

lemma mapIso_tprod[\[source\]](#)

```
@[simp]
lemma mapIso_tprod {c : X → T.Color} {d : Y → T.Color} (e : X ≈ Y) (h : c = d ∘ e)
  (f : (i : X) → T.ColorModule (c i)) : (T.mapIso e h) (PiTensorProduct.tprod R f) =
  (PiTensorProduct.tprod R (fun i => T.colorModuleCast (by rw [h]; simp) (f (e.symm i))))
```

abbrev PureTensor[\[source\]](#)

The type of pure tensors, i.e. of the form $v_1 \otimes v_2 \otimes v_3 \otimes \dots$

```
abbrev PureTensor (c : X → T.Color)
```

def elimPureTensor[\[source\]](#)

A pure tensor in $T.PureTensor$ ($Sum.elim\ cX\ cY$) constructed from a pure tensor in $T.PureTensor\ cX$ and a pure tensor in $T.PureTensor\ cY$.

```
def elimPureTensor (p : T.PureTensor cX) (q : T.PureTensor cY) : T.PureTensor (Sum.elim cX cY)
  )
```

lemma elimPureTensor_update_right[\[source\]](#)

```
@[simp]
lemma elimPureTensor_update_right (p : T.PureTensor cX) (q : T.PureTensor cY)
  (y : Y) (r : T.ColorModule (cY y)) : T.elimPureTensor p (Function.update q y r) =
  Function.update (T.elimPureTensor p q) (Sum.inr y) r
```

lemma elimPureTensor_update_left[\[source\]](#)

```
@[simp]
lemma elimPureTensor_update_left (p : T.PureTensor cX) (q : T.PureTensor cY)
  (x : X) (r : T.ColorModule (cX x)) : T.elimPureTensor (Function.update p x r) q =
  Function.update (T.elimPureTensor p q) (Sum.inl x) r
```

def inlPureTensor[\[source\]](#)

The projection of a pure tensor in $T.PureTensor$ ($Sum.elim\ cX\ cY$) onto a pure tensor in $T.PureTensor\ cX$.

```
def inlPureTensor (p : T.PureTensor (Sum.elim cX cY)) : T.PureTensor cX
```

def inrPureTensor[\[source\]](#)

The projection of a pure tensor in $\mathcal{T}.PureTensor$ ($Sum.elim\ cX\ cY$) onto a pure tensor in $\mathcal{T}.PureTensor\ cY$.

```
def inrPureTensor (p :  $\mathcal{T}.PureTensor\ (Sum.elim\ cX\ cY)$ ) :  $\mathcal{T}.PureTensor\ cY$ 
```

lemma inlPureTensor_update_left[\[source\]](#)

```
@[simp]
```

```
lemma inlPureTensor_update_left [DecidableEq (X  $\otimes$  Y)] (f :  $\mathcal{T}.PureTensor\ (Sum.elim\ cX\ cY)$ ) (x : X)
  (v1 :  $\mathcal{T}.ColorModule\ (Sum.elim\ cX\ cY\ (Sum.inl\ x))$ ) :
   $\mathcal{T}.inlPureTensor\ (Function.update\ f\ (Sum.inl\ x)\ v1) =$ 
   $Function.update\ (\mathcal{T}.inlPureTensor\ f)\ x\ v1$ 
```

lemma inrPureTensor_update_left[\[source\]](#)

```
@[simp]
```

```
lemma inrPureTensor_update_left [DecidableEq (X  $\otimes$  Y)] (f :  $\mathcal{T}.PureTensor\ (Sum.elim\ cX\ cY)$ ) (x : X)
  (v1 :  $\mathcal{T}.ColorModule\ (Sum.elim\ cX\ cY\ (Sum.inl\ x))$ ) :
   $\mathcal{T}.inrPureTensor\ (Function.update\ f\ (Sum.inl\ x)\ v1) = (\mathcal{T}.inrPureTensor\ f)$ 
```

lemma inrPureTensor_update_right[\[source\]](#)

```
@[simp]
```

```
lemma inrPureTensor_update_right [DecidableEq (X  $\otimes$  Y)] (f :  $\mathcal{T}.PureTensor\ (Sum.elim\ cX\ cY)$ ) (y : Y)
  (v1 :  $\mathcal{T}.ColorModule\ (Sum.elim\ cX\ cY\ (Sum.inr\ y))$ ) :
   $\mathcal{T}.inrPureTensor\ (Function.update\ f\ (Sum.inr\ y)\ v1) =$ 
   $Function.update\ (\mathcal{T}.inrPureTensor\ f)\ y\ v1$ 
```

lemma inlPureTensor_update_right[\[source\]](#)

```
@[simp]
```

```
lemma inlPureTensor_update_right [DecidableEq (X  $\otimes$  Y)] (f :  $\mathcal{T}.PureTensor\ (Sum.elim\ cX\ cY)$ ) (y : Y)
  (v1 :  $\mathcal{T}.ColorModule\ (Sum.elim\ cX\ cY\ (Sum.inr\ y))$ ) :
   $\mathcal{T}.inlPureTensor\ (Function.update\ f\ (Sum.inr\ y)\ v1) = (\mathcal{T}.inlPureTensor\ f)$ 
```

def elimPureTensorMullin[\[source\]](#)

The multilinear map taking pure tensors a $\mathcal{T}.PureTensor\ cX$ and a pure tensor in $\mathcal{T}.PureTensor\ cY$, and constructing a tensor in $\mathcal{T}.Tensor\ (Sum.elim\ cX\ cY)$.

```
def elimPureTensorMullin : MultilinearMap R (fun i =>  $\mathcal{T}.ColorModule\ (cX\ i)$ )
  (MultilinearMap R (fun x =>  $\mathcal{T}.ColorModule\ (cY\ x)$ ) ( $\mathcal{T}.Tensor\ (Sum.elim\ cX\ cY)$ )) where
```

def domCoproduct[\[source\]](#)

The multi-linear map taking a pure tensor in $\mathcal{T}.PureTensor\ (Sum.elim\ cX\ cY)$ and constructing a vector in $\mathcal{T}.Tensor\ cX\ \otimes[R]\ \mathcal{T}.Tensor\ cY$.

```
def domCoproduct : MultilinearMap R (fun x =>  $\mathcal{T}$ .ColorModule (Sum.elim cX cY x))
  ( $\mathcal{T}$ .Tensor cX  $\otimes$ [R]  $\mathcal{T}$ .Tensor cY) where
```

def tensoratorSymm

[\[source\]](#)

The linear map combining two tensors into a single tensor via the tensor product i.e.
 $v1 \ v2 \mapsto v1 \otimes v2$.

```
def tensoratorSymm :  $\mathcal{T}$ .Tensor cX  $\otimes$ [R]  $\mathcal{T}$ .Tensor cY  $\rightarrow_l$ [R]  $\mathcal{T}$ .Tensor (Sum.elim cX cY)
```

def tensorator

[\[source\]](#)

Splitting a tensor in $\mathcal{T}$.Tensor (Sum.elim cX cY) into the tensor product of two tensors.

```
def tensorator :  $\mathcal{T}$ .Tensor (Sum.elim cX cY)  $\rightarrow_l$ [R]  $\mathcal{T}$ .Tensor cX  $\otimes$ [R]  $\mathcal{T}$ .Tensor cY
```

def tensoratorEquiv

[\[source\]](#)

An equivalence formed by taking the tensor product of tensors.

```
def tensoratorEquiv (c : X  $\rightarrow$   $\mathcal{T}$ .Color) (d : Y  $\rightarrow$   $\mathcal{T}$ .Color) :
   $\mathcal{T}$ .Tensor c  $\otimes$ [R]  $\mathcal{T}$ .Tensor d  $\approx_l$ [R]  $\mathcal{T}$ .Tensor (Sum.elim c d)
```

lemma tensoratorEquiv_tmul_tprod

[\[source\]](#)

```
@[simp]
lemma tensoratorEquiv_tmul_tprod (p :  $\mathcal{T}$ .PureTensor cX) (q :  $\mathcal{T}$ .PureTensor cY) :
  ( $\mathcal{T}$ .tensoratorEquiv cX cY) ((PiTensorProduct.tprod R) p  $\otimes_t$ [R] (PiTensorProduct.tprod R) q)
  =
  (PiTensorProduct.tprod R) ( $\mathcal{T}$ .elimPureTensor p q)
```

lemma tensoratorEquiv_mapIso_cond

[\[source\]](#)

```
lemma tensoratorEquiv_mapIso_cond {e : X  $\approx$  Y} {e' : Z  $\approx$  Y} {e'' : W  $\approx$  X}
  (h : cX =  $\mathcal{T}$ . $\tau$   $\circ$  cY  $\circ$  e) (h' : cZ = cY  $\circ$  e') (h'' : bW = cX  $\circ$  e'') :
  Sum.elim bW cZ = Sum.elim cX cY  $\circ$   $\uparrow$ (e''.sumCongr e')
```

lemma tensoratorEquiv_mapIso

[\[source\]](#)

```
@[simp]
lemma tensoratorEquiv_mapIso (e : X  $\approx$  Y) (e' : Z  $\approx$  Y) (e'' : W  $\approx$  X)
  (h : cX =  $\mathcal{T}$ . $\tau$   $\circ$  cY  $\circ$  e) (h' : cZ = cY  $\circ$  e') (h'' : bW = cX  $\circ$  e'') :
  (TensorProduct.congr ( $\mathcal{T}$ .mapIso e'' h'') ( $\mathcal{T}$ .mapIso e' h'))  $\ll\gg_l$  ( $\mathcal{T}$ .tensoratorEquiv cX cY)
  = ( $\mathcal{T}$ .tensoratorEquiv bW cZ)
   $\ll\gg_l$  ( $\mathcal{T}$ .mapIso (Equiv.sumCongr e'' e') ( $\mathcal{T}$ .tensoratorEquiv_mapIso_cond h h' h''))
```

lemma tensoratorEquiv_mapIso_apply

[\[source\]](#)

```
@[simp]
lemma tensoratorEquiv_mapIso_apply (e : X  $\approx$  Y) (e' : Z  $\approx$  Y) (e'' : W  $\approx$  X)
  (h : cX =  $\mathcal{T}$ . $\tau$   $\circ$  cY  $\circ$  e) (h' : cZ = cY  $\circ$  e') (h'' : cW = cX  $\circ$  e'')
  (x :  $\mathcal{T}$ .Tensor cW  $\otimes$ [R]  $\mathcal{T}$ .Tensor cZ) :
```

```
( $\mathcal{T}$ .tensoratorEquiv cX cY) ((TensorProduct.congr ( $\mathcal{T}$ .mapIso e'' h'') ( $\mathcal{T}$ .mapIso e' h')) x) =
( $\mathcal{T}$ .mapIso (Equiv.sumCongr e'' e') ( $\mathcal{T}$ .tensoratorEquiv_mapIso_cond h h' h''))
(( $\mathcal{T}$ .tensoratorEquiv cW cZ) x)
```

lemma tensoratorEquiv_mapIso_tmul[\[source\]](#)

```
lemma tensoratorEquiv_mapIso_tmul (e : X  $\approx$  Y) (e' : Z  $\approx$  Y) (e'' : W  $\approx$  X)
(h : cX =  $\mathcal{T}$ . $\tau$   $\circ$  cY  $\circ$  e) (h' : cZ = cY  $\circ$  e') (h'' : cW = cX  $\circ$  e'')
(x :  $\mathcal{T}$ .Tensor cW) (y :  $\mathcal{T}$ .Tensor cZ) :
( $\mathcal{T}$ .tensoratorEquiv cX cY) (( $\mathcal{T}$ .mapIso e'' h'' x)  $\otimes_{\mathcal{T}[R]}$  ( $\mathcal{T}$ .mapIso e' h' y)) =
( $\mathcal{T}$ .mapIso (Equiv.sumCongr e'' e') ( $\mathcal{T}$ .tensoratorEquiv_mapIso_cond h h' h''))
(( $\mathcal{T}$ .tensoratorEquiv cW cZ) (x  $\otimes_{\mathcal{T}}$  y))
```

lemma contrDual_cast[\[source\]](#)

```
lemma contrDual_cast (h :  $\mu = \nu$ ) (x :  $\mathcal{T}$ .ColorModule  $\mu$ ) (y :  $\mathcal{T}$ .ColorModule ( $\mathcal{T}$ . $\tau$   $\mu$ )) :
 $\mathcal{T}$ .contrDual  $\mu$  (x  $\otimes_{\mathcal{T}[R]}$  y) =  $\mathcal{T}$ .contrDual  $\nu$  ( $\mathcal{T}$ .colorModuleCast h x  $\otimes_{\mathcal{T}[R]}$ 
 $\mathcal{T}$ .colorModuleCast (congrArg  $\mathcal{T}$ . $\tau$  h) y)
```

lemma contrDual_symm'[\[source\]](#)

$\mathcal{T}$.contrDual ($\mathcal{T}$. τ μ) in terms of $\mathcal{T}$.contrDual μ .

@[simp]

```
lemma contrDual_symm' ( $\mu$  :  $\mathcal{T}$ .Color) (x :  $\mathcal{T}$ .ColorModule ( $\mathcal{T}$ . $\tau$   $\mu$ ))
(y :  $\mathcal{T}$ .ColorModule ( $\mathcal{T}$ . $\tau$  ( $\mathcal{T}$ . $\tau$   $\mu$ ))) :  $\mathcal{T}$ .contrDual ( $\mathcal{T}$ . $\tau$   $\mu$ ) (x  $\otimes_{\mathcal{T}[R]}$  y) =
( $\mathcal{T}$ .contrDual  $\mu$ ) (( $\mathcal{T}$ .colorModuleCast ( $\mathcal{T}$ . $\tau$ _involutive  $\mu$ ) y)  $\otimes_{\mathcal{T}[R]}$  x)
```

lemma contrDual_symm_contrRightAux[\[source\]](#)

```
lemma contrDual_symm_contrRightAux (h :  $\nu = \eta$ ) :
( $\mathcal{T}$ .colorModuleCast h)  $\circ_l$  contrRightAux ( $\mathcal{T}$ .contrDual  $\mu$ ) =
contrRightAux ( $\mathcal{T}$ .contrDual ( $\mathcal{T}$ . $\tau$  ( $\mathcal{T}$ . $\tau$   $\mu$ )))  $\circ_l$ 
(TensorProduct.congr (
TensorProduct.congr ( $\mathcal{T}$ .colorModuleCast h) ( $\mathcal{T}$ .colorModuleCast ( $\mathcal{T}$ . $\tau$ _involutive  $\mu$ ).symm))
( $\mathcal{T}$ .colorModuleCast (( $\mathcal{T}$ . $\tau$ _involutive ( $\mathcal{T}$ . $\tau$   $\mu$ )).symm))).toLinearMap
```

lemma contrDual_symm_contrRightAux_apply_tmul[\[source\]](#)

```
lemma contrDual_symm_contrRightAux_apply_tmul (h :  $\nu = \eta$ )
(m :  $\mathcal{T}$ .ColorModule  $\nu$   $\otimes_{\mathcal{T}[R]}$   $\mathcal{T}$ .ColorModule  $\mu$ ) (x :  $\mathcal{T}$ .ColorModule ( $\mathcal{T}$ . $\tau$   $\mu$ )) :
 $\mathcal{T}$ .colorModuleCast h (contrRightAux ( $\mathcal{T}$ .contrDual  $\mu$ ) (m  $\otimes_{\mathcal{T}[R]}$  x)) =
contrRightAux ( $\mathcal{T}$ .contrDual ( $\mathcal{T}$ . $\tau$  ( $\mathcal{T}$ . $\tau$   $\mu$ ))) ((TensorProduct.congr
( $\mathcal{T}$ .colorModuleCast h) ( $\mathcal{T}$ .colorModuleCast ( $\mathcal{T}$ . $\tau$ _involutive  $\mu$ ).symm) (m))  $\otimes_{\mathcal{T}}$ 
( $\mathcal{T}$ .colorModuleCast ( $\mathcal{T}$ . $\tau$ _involutive ( $\mathcal{T}$ . $\tau$   $\mu$ )).symm x))
```

def decompEmbedSet[\[source\]](#)

The decomposition of a set into a direct sum based on the image of an injection.

```
def decompEmbedSet (f : Y  $\hookrightarrow$  X) :
X  $\approx$  {x // x  $\in$  (Finset.image f Finset.univ)c}  $\oplus$  Y
```

def decompEmbedColorLeft[\[source\]](#)

The restriction of a map from an indexing set to the space to the complement of the image of an embedding.

```
def decompEmbedColorLeft (c : X → T.Color) (f : Y ↪ X) :
  {x // x ∈ (Finset.image f Finset.univ)c} → T.Color
```

def decompEmbedColorRight[\[source\]](#)

The restriction of a map from an indexing set to the space to the image of an embedding.

```
def decompEmbedColorRight (c : X → T.Color) (f : Y ↪ X) : Y → T.Color
```

lemma decompEmbed_cond[\[source\]](#)

```
lemma decompEmbed_cond (c : X → T.Color) (f : Y ↪ X) : c =
  (Sum.elim (T.decompEmbedColorLeft c f) (T.decompEmbedColorRight c f)) ∘ decompEmbedSet f
```

def decompEmbed[\[source\]](#)

Decomposes a tensor into a tensor product of two tensors one which has indices in the image of the given embedding, and the other has indices in the complement of that image.

```
def decompEmbed (f : Y ↪ X) :
  T.Tensor cX ≈l[R] T.Tensor (T.decompEmbedColorLeft cX f) ⊗[R] T.Tensor (cX ∘ f)
```

2.12 HepLean.SpaceTime.LorentzTensor.Contraction

[HepLean/SpaceTime/LorentzTensor/Contraction.lean](#) on GitHub.

def contrDualLeft[\[source\]](#)

The contraction of a vector in $\mathcal{T}.ColorModule$ v with a vector in $\mathcal{T}.ColorModule$ $(\mathcal{T}.\tau v) \otimes_{[R]} \mathcal{T}.ColorModule \eta$ to form a vector in $\mathcal{T}.ColorModule \eta$.

```
def contrDualLeft {v η : T.Color} :
  T.ColorModule v ⊗[R] T.ColorModule (T.τ v) ⊗[R] T.ColorModule η →l[R] T.ColorModule η
```

def contrDualMid[\[source\]](#)

The contraction of a vector in $\mathcal{T}.ColorModule \mu \otimes_{[R]} \mathcal{T}.ColorModule v$ with a vector in $\mathcal{T}.ColorModule (\mathcal{T}.\tau v) \otimes_{[R]} \mathcal{T}.ColorModule \eta$ to form a vector in $\mathcal{T}.ColorModule \mu \otimes_{[R]} \mathcal{T}.ColorModule \eta$.

```
def contrDualMid {μ v η : T.Color} :
  (T.ColorModule μ ⊗[R] T.ColorModule v) ⊗[R] (T.ColorModule (T.τ v) ⊗[R] T.ColorModule η)
  →l[R]
  T.ColorModule μ ⊗[R] T.ColorModule η
```

def pairProd[\[source\]](#)

A linear map taking tensors mapped with the same index set to the product of paired tensors.

```
def pairProd :  $\mathcal{T}.$ Tensor cX  $\otimes$ [R]  $\mathcal{T}.$ Tensor cX2  $\rightarrow_l$ [R]
 $\bigotimes$ [R] x,  $\mathcal{T}.$ ColorModule (cX x)  $\otimes$ [R]  $\mathcal{T}.$ ColorModule (cX2 x)
```

lemma mkPiAlgebra_equiv[\[source\]](#)

```
lemma mkPiAlgebra_equiv (e : X  $\approx$  Y) :
  (PiTensorProduct.lift (MultilinearMap.mkPiAlgebra R X R)) =
  (PiTensorProduct.lift (MultilinearMap.mkPiAlgebra R Y R))  $\circ_l$ 
  (PiTensorProduct.reindex R _ e).toLinearMap
```

def contrAll'[\[source\]](#)

Given a tensor in $\mathcal{T}.$ Tensor cX and a tensor in $\mathcal{T}.$ Tensor ($\mathcal{T}.\tau \circ cX$), the element of R formed by contracting all of their indices.

```
def contrAll' :  $\mathcal{T}.$ Tensor cX  $\otimes$ [R]  $\mathcal{T}.$ Tensor ( $\mathcal{T}.\tau \circ cX$ )  $\rightarrow_l$ [R] R
```

lemma contrAll'_mapIso_cond[\[source\]](#)

```
lemma contrAll'_mapIso_cond {e : X  $\approx$  Y} (h : cX = cY  $\circ$  e) :
   $\mathcal{T}.\tau \circ cY = (\mathcal{T}.\tau \circ cX) \circ \uparrow e.$ symm
```

lemma contrAll'_mapIso[\[source\]](#)

```
@[simp]
lemma contrAll'_mapIso (e : X  $\approx$  Y) (h : c = cY  $\circ$  e) :
   $\mathcal{T}.$ contrAll'  $\circ_l$ 
  (TensorProduct.congr ( $\mathcal{T}.$ mapIso e h) (LinearEquiv.refl R _)).toLinearMap =
   $\mathcal{T}.$ contrAll'  $\circ_l$  (TensorProduct.congr (LinearEquiv.refl R _)
    ( $\mathcal{T}.$ mapIso e.symm ( $\mathcal{T}.$ contrAll'_mapIso_cond h))).toLinearMap
```

lemma contrAll'_mapIso_tmul[\[source\]](#)

```
@[simp]
lemma contrAll'_mapIso_tmul (e : X  $\approx$  Y) (h : c = cY  $\circ$  e) (x :  $\mathcal{T}.$ Tensor c)
  (y :  $\mathcal{T}.$ Tensor ( $\mathcal{T}.\tau \circ cY$ )) :  $\mathcal{T}.$ contrAll' (( $\mathcal{T}.$ mapIso e h) x  $\otimes_t$ [R] y) =
   $\mathcal{T}.$ contrAll' (x  $\otimes_t$ [R] ( $\mathcal{T}.$ mapIso e.symm ( $\mathcal{T}.$ contrAll'_mapIso_cond h) y))
```

def contrAll[\[source\]](#)

The contraction of all the indices of two tensors with dual colors.

```
def contrAll (e : X  $\approx$  Y) (h : cX =  $\mathcal{T}.\tau \circ cY \circ$  e) :  $\mathcal{T}.$ Tensor cX  $\otimes$ [R]  $\mathcal{T}.$ Tensor cY  $\rightarrow_l$ [R] R
```

lemma contrAll_symm_cond[\[source\]](#)

```
lemma contrAll_symm_cond {e : X  $\approx$  Y} (h : c =  $\mathcal{T}.\tau \circ cY \circ$  e) :
  cY =  $\mathcal{T}.\tau \circ c \circ \uparrow e.$ symm
```

lemma contrAll_mapIso_right_cond[\[source\]](#)

```
lemma contrAll_mapIso_right_cond {e : X ≈ Y} {e' : Z ≈ Y}
  (h : c =  $\mathcal{T}.\tau \circ cY \circ e$ ) (h' : cZ = cY ∘ e') : c =  $\mathcal{T}.\tau \circ cZ \circ \uparrow(e.trans\ e'.symm)$ 
```

lemma contrAll_mapIso_right_tmul[\[source\]](#)

```
@[simp]
lemma contrAll_mapIso_right_tmul (e : X ≈ Y) (e' : Z ≈ Y)
  (h : c =  $\mathcal{T}.\tau \circ cY \circ e$ ) (h' : cZ = cY ∘ e') (x :  $\mathcal{T}.Tensor\ c$ ) (z :  $\mathcal{T}.Tensor\ cZ$ ) :
   $\mathcal{T}.contrAll\ e\ h\ (x \otimes_t[R]\ \mathcal{T}.mapIso\ e'\ h'\ z) =$ 
   $\mathcal{T}.contrAll\ (e.trans\ e'.symm)\ (\mathcal{T}.contrAll\_mapIso\_right\_cond\ h\ h')\ (x \otimes_t[R]\ z)$ 
```

lemma contrAll_comp_mapIso_right[\[source\]](#)

```
@[simp]
lemma contrAll_comp_mapIso_right (e : X ≈ Y) (e' : Z ≈ Y)
  (h : c =  $\mathcal{T}.\tau \circ cY \circ e$ ) (h' : cZ = cY ∘ e') :  $\mathcal{T}.contrAll\ e\ h \circ_l$ 
  (TensorProduct.congr (LinearEquiv.refl R ( $\mathcal{T}.Tensor\ c$ )) ( $\mathcal{T}.mapIso\ e'\ h'$ )).toLinearMap
  =  $\mathcal{T}.contrAll\ (e.trans\ e'.symm)\ (\mathcal{T}.contrAll\_mapIso\_right\_cond\ h\ h')$ 
```

lemma contrAll_mapIso_left_cond[\[source\]](#)

```
lemma contrAll_mapIso_left_cond {e : X ≈ Y} {e' : Z ≈ X}
  (h : c =  $\mathcal{T}.\tau \circ cY \circ e$ ) (h' : cZ = c ∘ e') : cZ =  $\mathcal{T}.\tau \circ cY \circ \uparrow(e'.trans\ e)$ 
```

lemma contrAll_mapIso_left_tmul[\[source\]](#)

```
@[simp]
lemma contrAll_mapIso_left_tmul {e : X ≈ Y} {e' : Z ≈ X}
  (h : c =  $\mathcal{T}.\tau \circ cY \circ e$ ) (h' : cZ = c ∘ e') (x :  $\mathcal{T}.Tensor\ cZ$ ) (y :  $\mathcal{T}.Tensor\ cY$ ) :
   $\mathcal{T}.contrAll\ e\ h\ (\mathcal{T}.mapIso\ e'\ h'\ x \otimes_t[R]\ y) =$ 
   $\mathcal{T}.contrAll\ (e'.trans\ e)\ (\mathcal{T}.contrAll\_mapIso\_left\_cond\ h\ h')\ (x \otimes_t[R]\ y)$ 
```

lemma contrAll_mapIso_left[\[source\]](#)

```
@[simp]
lemma contrAll_mapIso_left {e : X ≈ Y} {e' : Z ≈ X}
  (h : c =  $\mathcal{T}.\tau \circ cY \circ e$ ) (h' : cZ = c ∘ e') :
   $\mathcal{T}.contrAll\ e\ h \circ_l$ 
  (TensorProduct.congr ( $\mathcal{T}.mapIso\ e'\ h'$ ) (LinearEquiv.refl R ( $\mathcal{T}.Tensor\ cY$ ))).toLinearMap
  =  $\mathcal{T}.contrAll\ (e'.trans\ e)\ (\mathcal{T}.contrAll\_mapIso\_left\_cond\ h\ h')$ 
```

def contrAllLeft[\[source\]](#)

The linear map from $\mathcal{T}.Tensor\ cX \otimes[R]\ \mathcal{T}.Tensor\ cY \otimes[R]\ \mathcal{T}.Tensor\ cZ$ to $\mathcal{T}.Tensor\ cZ$ obtained by contracting all indices in $\mathcal{T}.Tensor\ cX$ and $\mathcal{T}.Tensor\ cY$, given a proof that this is possible.

```
def contrAllLeft (e : X ≈ Y) (h : cX =  $\mathcal{T}.\tau \circ cY \circ e$ ) :
   $\mathcal{T}.Tensor\ cX \otimes[R]\ \mathcal{T}.Tensor\ cY \otimes[R]\ \mathcal{T}.Tensor\ cZ \rightarrow_l[R]\ \mathcal{T}.Tensor\ cZ$ 
```


def contrAllMid[\[source\]](#)

The linear map from $(\mathcal{T}.Tensor\ cW \otimes [R] \mathcal{T}.Tensor\ cX) \otimes [R] (\mathcal{T}.Tensor\ cY \otimes [R] \mathcal{T}.Tensor\ cZ)$ to $\mathcal{T}.Tensor\ cW \otimes [R] \mathcal{T}.Tensor\ cZ$ obtained by contracting all indices of the tensors in the middle.

```
def contrAllMid (e : X ≈ Y) (h : cX =  $\mathcal{T}.\tau \circ cY \circ e$ ) :
  ( $\mathcal{T}.Tensor\ cW \otimes [R] \mathcal{T}.Tensor\ cX$ )  $\otimes [R]$  ( $\mathcal{T}.Tensor\ cY \otimes [R] \mathcal{T}.Tensor\ cZ$ )  $\rightarrow_l [R]$ 
   $\mathcal{T}.Tensor\ cW \otimes [R] \mathcal{T}.Tensor\ cZ$ 
```

def contrElim[\[source\]](#)

The linear map from $\mathcal{T}.Tensor\ (Sum.elim\ cW\ cX) \otimes [R] \mathcal{T}.Tensor\ (Sum.elim\ cY\ cZ)$ to $\mathcal{T}.Tensor\ (Sum.elim\ cW\ cZ)$ formed by contracting the indices specified by cX and cY , which are assumed to be dual.

```
def contrElim (e : X ≈ Y) (h : cX =  $\mathcal{T}.\tau \circ cY \circ e$ ) :
   $\mathcal{T}.Tensor\ (Sum.elim\ cW\ cX) \otimes [R] \mathcal{T}.Tensor\ (Sum.elim\ cY\ cZ) \rightarrow_l [R] \mathcal{T}.Tensor\ (Sum.elim\ cW\ cZ)$ 
```

lemma contrAll_rep[\[source\]](#)

```
@[simp]
lemma contrAll_rep {c : X →  $\mathcal{T}.Color$ } {d : Y →  $\mathcal{T}.Color$ } (e : X ≈ Y) (h : c =  $\mathcal{T}.\tau \circ d \circ e$ ) (g :
  G) :
   $\mathcal{T}.contrAll\ e\ h \circ_l (TensorProduct.map\ (\mathcal{T}.rep\ g)\ (\mathcal{T}.rep\ g)) = \mathcal{T}.contrAll\ e\ h$ 
```

lemma contrAll_rep_apply[\[source\]](#)

```
@[simp]
lemma contrAll_rep_apply {c : X →  $\mathcal{T}.Color$ } {d : Y →  $\mathcal{T}.Color$ } (e : X ≈ Y) (h : c =  $\mathcal{T}.\tau \circ d \circ e$ )
  (g : G) (x :  $\mathcal{T}.Tensor\ c \otimes \mathcal{T}.Tensor\ d$ ) :
   $\mathcal{T}.contrAll\ e\ h\ (TensorProduct.map\ (\mathcal{T}.rep\ g)\ (\mathcal{T}.rep\ g)\ x) = \mathcal{T}.contrAll\ e\ h\ x$ 
```

lemma contrAll_rep_tmul[\[source\]](#)

```
@[simp]
lemma contrAll_rep_tmul {c : X →  $\mathcal{T}.Color$ } {d : Y →  $\mathcal{T}.Color$ } (e : X ≈ Y) (h : c =  $\mathcal{T}.\tau \circ d \circ e$ )
  (g : G) (x :  $\mathcal{T}.Tensor\ c$ ) (y :  $\mathcal{T}.Tensor\ d$ ) :
   $\mathcal{T}.contrAll\ e\ h\ ((g \cdot x) \otimes_t [R] (g \cdot y)) = \mathcal{T}.contrAll\ e\ h\ (x \otimes_t [R] y)$ 
```

lemma contr_cond[\[source\]](#)

```
lemma contr_cond (e : (C ⊗ C) ⊗ P ≈ X) :
  cX = Sum.elim (Sum.elim (cX ∘  $\uparrow e \circ Sum.inl \circ Sum.inl$ ) (cX ∘  $\uparrow e \circ Sum.inl \circ Sum.inr$ )) (
    cX ∘  $\uparrow e \circ Sum.inr$ ) ∘  $\uparrow e.symm$ 
```

def contr[\[source\]](#)

Contraction of indices based on an equivalence $(C \otimes C) \otimes P \approx X$. The indices in C are contracted pair-wise, whilst the indices in P are preserved.

```
def contr (e : (C ⊗ C) ⊗ P ≈ X)
  (h : cX ∘ e ∘ Sum.inl ∘ Sum.inl =  $\mathcal{T}.\tau \circ cX \circ e \circ Sum.inl \circ Sum.inr$ ) :
```

$$\mathcal{T}.\text{Tensor } cX \rightarrow_l[R] \mathcal{T}.\text{Tensor } (cX \circ e \circ \text{Sum.inr})$$
lemma contr_equivariant[\[source\]](#)

The contraction of indices via `contr` is equivariant.

```
@[simp]
lemma contr_equivariant (e : (C ⊗ C) ⊗ P ≈ X)
  (h : cX ∘ e ∘ Sum.inl ∘ Sum.inl =  $\mathcal{T}.\tau$  ∘ cX ∘ e ∘ Sum.inl ∘ Sum.inr)
  (g : G) (x :  $\mathcal{T}.\text{Tensor } cX$ ) :  $\mathcal{T}.\text{contr } e \ h \ (g \cdot x) = g \cdot \mathcal{T}.\text{contr } e \ h \ x$ 
```

2.13 HepLean.SpaceTime.LorentzTensor.Fin

[HepLean/SpaceTime/LorentzTensor/Fin.lean](#) on GitHub.

def finCast[\[source\]](#)

Casting a tensor defined on $\text{Fin } n$ to $\text{Fin } m$ where $n = m$.

```
@[simp]
def finCast (h : n = m) (hc : cn = cm ∘ Fin.castOrderIso h) :  $\mathcal{T}.\text{Tensor } cn \approx_l[R] \mathcal{T}.\text{Tensor } cm$ 
```

def finSumEquiv[\[source\]](#)

An equivalence between $\mathcal{T}.\text{Tensor } cn \otimes[R] \mathcal{T}.\text{Tensor } cm$ indexed by $\text{Fin } n$ and $\text{Fin } m$, and $\mathcal{T}.\text{Tensor } (\text{Sum.elim } cn \ cm \circ \text{finSumFinEquiv.symm})$ indexed by $\text{Fin } (n + m)$.

```
@[simp]
def finSumEquiv :  $\mathcal{T}.\text{Tensor } cn \otimes[R] \mathcal{T}.\text{Tensor } cm \approx_l[R] \mathcal{T}.\text{Tensor } (\text{Sum.elim } cn \ cm \circ \text{finSumFinEquiv.symm})$ 
```

def tensorSingletonEquiv[\[source\]](#)

Tensors on $\text{Fin } 1$ are equivalent to a constant P_i tensor product.

```
def tensorSingletonEquiv :  $\mathcal{T}.\text{Tensor } ![\mu] \approx_l[R] \bigotimes[R] \_ : (\text{Fin } 1), \mathcal{T}.\text{ColorModule } \mu$ 
```

def vecAsTensor[\[source\]](#)

An equivalence between the ColorModule for a color and a $\text{Fin } 1$ tensor with that color.

```
def vecAsTensor :  $\mathcal{T}.\text{ColorModule } \mu \approx_l[R] \mathcal{T}.\text{Tensor } ![\mu]$ 
```

lemma vecAsTensor_equivariant[\[source\]](#)

The equivalence `vecAsTensor` is equivariant with respect to `MulActionTensor`-actions.

```
@[simp]
lemma vecAsTensor_equivariant (g : G) (v :  $\mathcal{T}.\text{ColorModule } \mu$ ) :
   $\mathcal{T}.\text{vecAsTensor } (\text{repColorModule } \mu \ g \ v) = g \cdot \mathcal{T}.\text{vecAsTensor } v$ 
```

2.14 HepLean.SpaceTime.LorentzTensor.MulActionTensor

[HepLean/SpaceTime/LorentzTensor/MulActionTensor.lean](#) on GitHub.

class MulActionTensor

[\[source\]](#)

A multiplicative action on a tensor structure (e.g. the action of the Lorentz group on real Lorentz tensors).

```
class MulActionTensor (G : Type) [Monoid G] (T : TensorStructure R) where
  repColorModule : (μ : T.Color) → Representation R G (T.ColorModule μ)
  contrDual_inv : ∀ μ g, T.contrDual μ ∘l
    TensorProduct.map (repColorModule μ g) (repColorModule (T.τ μ) g) = T.contrDual μ
  metric_inv : ∀ μ g, (TensorProduct.map (repColorModule μ g) (repColorModule μ g)) (T.metric
    μ) =
    T.metric μ
```

def compHom

[\[source\]](#)

The MulActionTensor defined by restriction along a group homomorphism.

```
def compHom (f : H →* G) : MulActionTensor H T where
```

def trivial

[\[source\]](#)

The trivial MulActionTensor defined via trivial representations.

```
def trivial : MulActionTensor G T where
```

lemma contrDual_equivariant_tmul

[\[source\]](#)

```
@[simp]
lemma contrDual_equivariant_tmul (g : G) (x : T.ColorModule μ) (y : T.ColorModule (T.τ μ)) :
  (T.contrDual μ ((repColorModule μ g x) ⊗t[R] (repColorModule (T.τ μ) g y))) =
  T.contrDual μ (x ⊗t[R] y)
```

lemma colorModuleCast_equivariant_apply

[\[source\]](#)

```
@[simp]
lemma colorModuleCast_equivariant_apply (h : μ = ν) (g : G) (x : T.ColorModule μ) :
  (T.colorModuleCast h) (repColorModule μ g x) =
  (repColorModule ν g) (T.colorModuleCast h x)
```

lemma contrRightAux_contrDual_equivariant_tmul

[\[source\]](#)

```
@[simp]
lemma contrRightAux_contrDual_equivariant_tmul (g : G) (m : T.ColorModule ν ⊗t[R] T.ColorModule
  μ)
  (x : T.ColorModule (T.τ μ)) : (contrRightAux (T.contrDual μ))
  ((TensorProduct.map (repColorModule ν g) (repColorModule μ g) m) ⊗t[R]
  (repColorModule (T.τ μ) g x)) =
  repColorModule ν g ((contrRightAux (T.contrDual μ)) (m ⊗t[R] x))
```

def rep[\[source\]](#)

The representation of the group G on the vector space of tensors.

```
def rep : Representation R G (T.Tensor cX) where
```

lemma repColorModule_colorModuleCast[\[source\]](#)

```
@[simp]
```

```
lemma repColorModule_colorModuleCast (h : μ = ν) (g : G) :
  (repColorModule ν g) ∘l (T.colorModuleCast h).toLinearMap =
  (T.colorModuleCast h).toLinearMap ∘l (repColorModule μ g)
```

lemma rep_mapIso[\[source\]](#)

```
@[simp]
```

```
lemma rep_mapIso (e : X ≈ Y) (h : cX = cY ∘ e) (g : G) :
  (T.rep g) ∘l (T.mapIso e h).toLinearMap = (T.mapIso e h).toLinearMap ∘l T.rep g
```

lemma rep_mapIso_apply[\[source\]](#)

```
@[simp]
```

```
lemma rep_mapIso_apply (e : X ≈ Y) (h : cX = cY ∘ e) (g : G) (x : T.Tensor cX) :
  (T.mapIso e h) (g • x) = g • (T.mapIso e h x)
```

lemma rep_tprod[\[source\]](#)

```
@[simp]
```

```
lemma rep_tprod (g : G) (f : (i : X) → T.ColorModule (cX i)) :
  g • (PiTensorProduct.tprod R f) = PiTensorProduct.tprod R (fun x =>
    repColorModule (cX x) g (f x))
```

lemma tensoratorEquiv_equivariant[\[source\]](#)

```
lemma tensoratorEquiv_equivariant (g : G) :
```

```
(T.tensoratorEquiv cX cY) ∘l (TensorProduct.map (T.rep g) (T.rep g)) = T.rep g ∘l
(T.tensoratorEquiv cX cY).toLinearMap
```

lemma tensoratorEquiv_equivariant_apply[\[source\]](#)

```
@[simp]
```

```
lemma tensoratorEquiv_equivariant_apply (g : G) (x : T.Tensor cX ⊗[R] T.Tensor cY) :
  (T.tensoratorEquiv cX cY) ((TensorProduct.map (T.rep g) (T.rep g)) x)
  = (T.rep g) ((T.tensoratorEquiv cX cY) x)
```

lemma rep_tensoratorEquiv_tmul[\[source\]](#)

```
lemma rep_tensoratorEquiv_tmul (g : G) (x : T.Tensor cX) (y : T.Tensor cY) :
```

```
(T.tensoratorEquiv cX cY) ((g • x) ⊗t[R] (g • y)) =
  g • ((T.tensoratorEquiv cX cY) (x ⊗t[R] y))
```

lemma rep_tensoratorEquiv_symm[\[source\]](#)

```

lemma rep_tensoratorEquiv_symm (g : G) :
  (T.tensoratorEquiv cX cY).symm ∘l T.rep g = (TensorProduct.map (T.rep g) (T.rep g)) ∘l
  (T.tensoratorEquiv cX cY).symm.toLinearMap

```

lemma rep_tensoratorEquiv_symm_apply[\[source\]](#)

```

@[simp]
lemma rep_tensoratorEquiv_symm_apply (g : G) (x : T.Tensor (Sum.elim cX cY)) :
  (T.tensoratorEquiv cX cY).symm ((T.rep g) x) =
  (TensorProduct.map (T.rep g) (T.rep g)) ((T.tensoratorEquiv cX cY).symm x)

```

lemma rep_lid[\[source\]](#)

```

@[simp]
lemma rep_lid (g : G) : TensorProduct.lid R (T.Tensor cX) ∘l
  (TensorProduct.map (LinearMap.id) (T.rep g)) = (T.rep g) ∘l
  (TensorProduct.lid R (T.Tensor cX)).toLinearMap

```

lemma rep_lid_apply[\[source\]](#)

```

@[simp]
lemma rep_lid_apply (g : G) (x : R ⊗[R] T.Tensor cX) :
  (TensorProduct.lid R (T.Tensor cX)) ((TensorProduct.map (LinearMap.id) (T.rep g)) x) =
  (T.rep g) ((TensorProduct.lid R (T.Tensor cX)).toLinearMap x)

```

2.15 HepLean.SpaceTime.LorentzTensor.Real.Basic

[HepLean/SpaceTime/LorentzTensor/Real/Basic.lean](#) on GitHub.

inductive ColorType[\[source\]](#)

The type colors for real Lorentz tensors.

```

inductive ColorType
| up
| down

```

def realLorentzTensor[\[source\]](#)

The LorentzTensorStructure associated with real Lorentz tensors.

```

def realLorentzTensor (d : ℕ) : TensorStructure ℝ where

```

instance MulActionTensor (LorentzGroup d) (realLorentzTensor d)[\[source\]](#)

The action of the Lorentz group on real Lorentz tensors.

```

instance : MulActionTensor (LorentzGroup d) (realLorentzTensor d) where

```

2.16 HepLean.SpaceTime.LorentzTensor.RisingLowering

[HepLean/SpaceTime/LorentzTensor/RisingLowering.lean](#) on GitHub.

lemma unit_lhs_eq[\[source\]](#)

```

lemma unit_lhs_eq (x :  $\mathcal{T}$ .ColorModule  $\mu$ ) (y :  $\mathcal{T}$ .ColorModule ( $\mathcal{T}.\tau \mu$ )  $\otimes_{[R]} \mathcal{T}$ .ColorModule  $\mu$ ) :
  contrLeftAux ( $\mathcal{T}$ .contrDual  $\mu$ ) (x  $\otimes_{\tau[R]} y$ ) =
  (contrRightAux ( $\mathcal{T}$ .contrDual ( $\mathcal{T}.\tau \mu$ ))) ((TensorProduct.comm R _ _) y
   $\otimes_{\tau[R]} (\mathcal{T}.colorModuleCast (\mathcal{T}.\tau\_involutive \mu).symm) x$ )

```

lemma unit_lid[\[source\]](#)

```

@[simp]
lemma unit_lid : (contrRightAux ( $\mathcal{T}$ .contrDual ( $\mathcal{T}.\tau \mu$ ))) ((TensorProduct.comm R _ _) ( $\mathcal{T}$ .unit  $\mu$ )
   $\otimes_{\tau[R]} (\mathcal{T}.colorModuleCast (\mathcal{T}.\tau\_involutive \mu).symm) x$ ) = x

```

lemma metric_cast[\[source\]](#)

```

@[simp]
lemma metric_cast (h :  $\mu = \nu$ ) :
  (TensorProduct.congr ( $\mathcal{T}.colorModuleCast h$ ) ( $\mathcal{T}.colorModuleCast h$ )) ( $\mathcal{T}$ .metric  $\mu$ ) =
   $\mathcal{T}$ .metric  $\nu$ 

```

lemma metric_contrRight_unit[\[source\]](#)

```

@[simp]
lemma metric_contrRight_unit ( $\mu$  :  $\mathcal{T}$ .Color) (x :  $\mathcal{T}$ .ColorModule  $\mu$ ) :
  (contrRightAux ( $\mathcal{T}$ .contrDual  $\mu$ )) ( $\mathcal{T}$ .metric  $\mu$   $\otimes_{\tau[R]}$ 
  ((contrRightAux ( $\mathcal{T}$ .contrDual ( $\mathcal{T}.\tau \mu$ )))
  ( $\mathcal{T}$ .metric ( $\mathcal{T}.\tau \mu$ )  $\otimes_{\tau[R]} (\mathcal{T}.colorModuleCast (\mathcal{T}.\tau\_involutive \mu).symm) x$ ))) = x

```

def dualizeSymm[\[source\]](#)

*Takes a vector with index with dual color to a vector with index the underlying color.
Obtained by contraction with the metric.*

```

def dualizeSymm ( $\mu$  :  $\mathcal{T}$ .Color) :  $\mathcal{T}$ .ColorModule ( $\mathcal{T}.\tau \mu$ )  $\rightarrow_{[R]} \mathcal{T}$ .ColorModule  $\mu$ 

```

def dualizeFun[\[source\]](#)

Takes a vector to a vector with the dual color index. Obtained by contraction with the metric.

```

def dualizeFun ( $\mu$  :  $\mathcal{T}$ .Color) :  $\mathcal{T}$ .ColorModule  $\mu \rightarrow_{[R]} \mathcal{T}$ .ColorModule ( $\mathcal{T}.\tau \mu$ )

```

def dualizeModule[\[source\]](#)

Equivalence between the module with a color μ and the module with color $\mathcal{T}.\tau \mu$ obtained by contraction with the metric.

```

def dualizeModule ( $\mu$  :  $\mathcal{T}$ .Color) :  $\mathcal{T}$ .ColorModule  $\mu \approx_{[R]} \mathcal{T}$ .ColorModule ( $\mathcal{T}.\tau \mu$ )

```

lemma dualizeModule_equivariant[\[source\]](#)

```

@[simp]
lemma dualizeModule_equivariant (g : G) :
  ( $\mathcal{T}$ .dualizeModule  $\mu$ )  $\circ_l ((MulActionTensor.repColorModule \mu) g)$  =

```

```
(MulActionTensor.repColorModule (T.τ μ) g) ∘l (T.dualizeModule μ)
```

lemma dualizeModule_equivariant_apply

[\[source\]](#)

```
@[simp]
lemma dualizeModule_equivariant_apply (g : G) (x : T.ColorModule μ) :
  (T.dualizeModule μ) ((MulActionTensor.repColorModule μ) g x) =
  (MulActionTensor.repColorModule (T.τ μ) g) (T.dualizeModule μ x)
```

def dualizeAll

[\[source\]](#)

Dualizes the color of all indicies of a tensor by contraction with the metric.

```
def dualizeAll : T.Tensor cX ≈l[R] T.Tensor (T.τ ∘ cX)
```

lemma dualizeAll_equivariant

[\[source\]](#)

```
@[simp]
lemma dualizeAll_equivariant (g : G) : (T.dualizeAll.toLinearMap) ∘l (@rep R _ G _ T _ X cX g)
  = T.rep g ∘l (T.dualizeAll.toLinearMap)
```

lemma dualize_cond

[\[source\]](#)

```
lemma dualize_cond (e : C ⊕ P ≈ X) :
  cX = Sum.elim (cX ∘ e ∘ Sum.inl) (cX ∘ e ∘ Sum.inr) ∘ e.symm
```

lemma dualize_cond'

[\[source\]](#)

```
lemma dualize_cond' (e : C ⊕ P ≈ X) :
  Sum.elim (T.τ ∘ cX ∘ ↑e ∘ Sum.inl) (cX ∘ ↑e ∘ Sum.inr) =
  (Sum.elim (T.τ ∘ cX ∘ ↑e ∘ Sum.inl) (cX ∘ ↑e ∘ Sum.inr) ∘ ↑e.symm) ∘ ↑e
```

def dualize

[\[source\]](#)

Given an equivalence $C \oplus P \approx X$ dualizes those indices of a tensor which correspond to C whilst leaving the indices P invariant.

```
def dualize (e : C ⊕ P ≈ X) : T.Tensor cX ≈l[R]
  T.Tensor (Sum.elim (T.τ ∘ cX ∘ e ∘ Sum.inl) (cX ∘ e ∘ Sum.inr) ∘ e.symm)
```

lemma dualize_equivariant_apply

[\[source\]](#)

Dualizing indices is equivariant with respect to the group action. This is the applied version of this statement.

```
@[simp]
lemma dualize_equivariant_apply (e : C ⊕ P ≈ X) (g : G) (x : T.Tensor cX) :
  T.dualize e (g • x) = g • (T.dualize e x)
```

2.17 HepLean.SpaceTime.LorentzVector.Basic

[HepLean/SpaceTime/LorentzVector/Basic.lean](#) on GitHub.

def LorentzVector[\[source\]](#)*The type of (contravariant) Lorentz Vectors in d -space dimensions.***def** LorentzVector : Type**instance AddCommMonoid (LorentzVector d)**[\[source\]](#)*An instance of an additive commutative monoid on LorentzVector.***instance** : AddCommMonoid (LorentzVector d)**instance Module $\mathbb{R}$ (LorentzVector d)**[\[source\]](#)*An instance of a module on LorentzVector.***noncomputable instance** : Module $\mathbb{R}$ (LorentzVector d)**instance AddCommGroup (LorentzVector d)**[\[source\]](#)**instance** : AddCommGroup (LorentzVector d)**instance TopologicalSpace (LorentzVector d)**[\[source\]](#)*The structure of a topological space LorentzVector d .***instance** : TopologicalSpace (LorentzVector d)**def space**[\[source\]](#)*The space components.*

@[simp]

def space : EuclideanSpace $\mathbb{R}$ (Fin d)**def time**[\[source\]](#)*The time component.*

@[simp]

def time : $\mathbb{R}$ **def stdBasis**[\[source\]](#)*The standard basis of LorentzVector indexed by Fin 1 $\oplus$ Fin (d).*

@[sims!]

noncomputable def stdBasis : Basis (Fin 1 $\oplus$ Fin (d)) $\mathbb{R}$ (LorentzVector d)**lemma stdBasis_apply**[\[source\]](#)**lemma** stdBasis_apply (μ v : Fin 1 $\oplus$ Fin d) : e μ v = **if** μ = v **then** 1 **else** 0

lemma decomp_stdBasis[\[source\]](#)

```
lemma decomp_stdBasis (v : LorentzVector d) :  $\sum i, v\ i \cdot e\ i = v$ 
```

lemma decomp_stdBasis'[\[source\]](#)

```
@[simp]
lemma decomp_stdBasis' (v : LorentzVector d) :
  v (Sum.inl 0) • e (Sum.inl 0) +  $\sum a_2 : \text{Fin } d, v\ (\text{Sum.inr } a_2) \cdot e\ (\text{Sum.inr } a_2) = v$ 
```

abbrev timeVec[\[source\]](#)

The standard unit time vector.

```
noncomputable abbrev timeVec : (LorentzVector d)
```

lemma timeVec_space[\[source\]](#)

```
lemma timeVec_space : (@timeVec d).space = 0
```

lemma timeVec_time[\[source\]](#)

```
lemma timeVec_time: (@timeVec d).time = 1
```

def spaceReflectionLin[\[source\]](#)

The reflection of space as a linear map.

```
@[simps!]
def spaceReflectionLin : LorentzVector d →ℓ[ℝ] LorentzVector d where
```

def spaceReflection[\[source\]](#)

The reflection of space.

```
@[simp]
def spaceReflection : LorentzVector d
```

lemma spaceReflection_space[\[source\]](#)

```
lemma spaceReflection_space : v.spaceReflection.space = - v.space
```

lemma spaceReflection_time[\[source\]](#)

```
lemma spaceReflection_time : v.spaceReflection.time = v.time
```

2.18 HepLean.SpaceTime.LorentzVector.Contraction

[HepLean/SpaceTime/LorentzVector/Contraction.lean](#) on GitHub.

def contrUpDownBi[\[source\]](#)

The bi-linear map defining the contraction of a contravariant Lorentz vector and a covariant Lorentz vector.

```
def contrUpDownBi : LorentzVector d →l[ℝ] CovariantLorentzVector d →l[ℝ] ℝ where
```

def contrUpDown[\[source\]](#)

The linear map defining the contraction of a contravariant Lorentz vector and a covariant Lorentz vector.

```
def contrUpDown : LorentzVector d ⊗ℝ CovariantLorentzVector d →l[ℝ] ℝ
```

lemma contrUpDown_tmul_eq_dotProduct[\[source\]](#)

```
lemma contrUpDown_tmul_eq_dotProduct {x : LorentzVector d} {y : CovariantLorentzVector d} :  
  contrUpDown (x ⊗ℝ y) = x ·v y
```

lemma contrUpDown_stdBasis_left[\[source\]](#)

```
@[simp]  
lemma contrUpDown_stdBasis_left (x : LorentzVector d) (i : Fin 1 ⊗ Fin d) :  
  contrUpDown (x ⊗ℝ (CovariantLorentzVector.stdBasis i)) = x i
```

lemma contrUpDown_stdBasis_right[\[source\]](#)

```
@[simp]  
lemma contrUpDown_stdBasis_right (x : CovariantLorentzVector d) (i : Fin 1 ⊗ Fin d) :  
  contrUpDown (e i ⊗ℝ x) = x i
```

def contrDownUp[\[source\]](#)

The linear map defining the contraction of a covariant Lorentz vector and a contravariant Lorentz vector.

```
def contrDownUp : CovariantLorentzVector d ⊗ℝ LorentzVector d →l[ℝ] ℝ
```

lemma contrDownUp_tmul_eq_dotProduct[\[source\]](#)

```
lemma contrDownUp_tmul_eq_dotProduct {x : CovariantLorentzVector d} {y : LorentzVector d} :  
  contrDownUp (x ⊗ℝ y) = x ·v y
```

def unitUp[\[source\]](#)

The unit of the contraction of contravariant Lorentz vector and a covariant Lorentz vector.

```
def unitUp : CovariantLorentzVector d ⊗ℝ LorentzVector d
```

lemma unitUp_rid[\[source\]](#)

```
lemma unitUp_rid (x : LorentzVector d) :
  TensorStructure.contrLeftAux contrUpDown (x ⊗t [ℝ] unitUp) = x
```

def unitDown[\[source\]](#)

The unit of the contraction of covariant Lorentz vector and a contravariant Lorentz vector.

```
def unitDown : LorentzVector d ⊗ [ℝ] CovariantLorentzVector d
```

lemma unitDown_rid[\[source\]](#)

```
lemma unitDown_rid (x : CovariantLorentzVector d) :
  TensorStructure.contrLeftAux contrDownUp (x ⊗t [ℝ] unitDown) = x
```

lemma contrUpDown_invariant_lorentzAction[\[source\]](#)

```
@[simp]
lemma contrUpDown_invariant_lorentzAction : @contrUpDown d ((LorentzVector.rep g) x ⊗t [ℝ]
  (CovariantLorentzVector.rep g) y) = contrUpDown (x ⊗t [ℝ] y)
```

lemma contrDownUp_invariant_lorentzAction[\[source\]](#)

```
@[simp]
lemma contrDownUp_invariant_lorentzAction : @contrDownUp d ((CovariantLorentzVector.rep g) x
  ⊗t [ℝ]
  (LorentzVector.rep g) y) = contrDownUp (x ⊗t [ℝ] y)
```

def asTenProd[\[source\]](#)

The metric tensor as an element of LorentzVector d ⊗ [ℝ] LorentzVector d.

```
def asTenProd : LorentzVector d ⊗ [ℝ] LorentzVector d
```

lemma asTenProd_diag[\[source\]](#)

```
lemma asTenProd_diag :
  @asTenProd d = ∑ μ, η μ μ • (LorentzVector.stdBasis μ ⊗t [ℝ] LorentzVector.stdBasis μ)
```

def asCoTenProd[\[source\]](#)

The metric tensor as an element of CovariantLorentzVector d ⊗ [ℝ] CovariantLorentzVector d.

```
def asCoTenProd : CovariantLorentzVector d ⊗ [ℝ] CovariantLorentzVector d
```

lemma asCoTenProd_diag[\[source\]](#)

```
lemma asCoTenProd_diag :
  @asCoTenProd d = ∑ μ, η μ μ • (CovariantLorentzVector.stdBasis μ ⊗t [ℝ]
  CovariantLorentzVector.stdBasis μ)
```

lemma contrLeft_asTenProd[\[source\]](#)

```
@[simp]
lemma contrLeft_asTenProd (x : CovariantLorentzVector d) :
  contrLeftAux contrDownUp (x ⊗ₜ[ℝ] asTenProd) =
  ∑ μ, ((η μ μ * x μ) • LorentzVector.stdBasis μ)
```

lemma contrLeft_asCoTenProd[\[source\]](#)

```
@[simp]
lemma contrLeft_asCoTenProd (x : LorentzVector d) :
  contrLeftAux contrUpDown (x ⊗ₜ[ℝ] asCoTenProd) =
  ∑ μ, ((η μ μ * x μ) • CovariantLorentzVector.stdBasis μ)
```

lemma asTenProd_contr_asCoTenProd[\[source\]](#)

```
@[simp]
lemma asTenProd_contr_asCoTenProd :
  (contrMidAux (contrUpDown) (asTenProd ⊗ₜ[ℝ] asCoTenProd))
  = TensorProduct.comm ℝ _ _ (@unitUp d)
```

lemma asCoTenProd_contr_asTenProd[\[source\]](#)

```
@[simp]
lemma asCoTenProd_contr_asTenProd :
  (contrMidAux (contrDownUp) (asCoTenProd ⊗ₜ[ℝ] asTenProd))
  = TensorProduct.comm ℝ _ _ (@unitDown d)
```

lemma asTenProd_invariant[\[source\]](#)

```
lemma asTenProd_invariant (g : LorentzGroup d) :
  TensorProduct.map (LorentzVector.rep g) (LorentzVector.rep g) asTenProd = asTenProd
```

lemma asCoTenProd_invariant[\[source\]](#)

```
lemma asCoTenProd_invariant (g : LorentzGroup d) :
  TensorProduct.map (CovariantLorentzVector.rep g) (CovariantLorentzVector.rep g)
  asCoTenProd = asCoTenProd
```

2.19 HepLean.SpaceTime.LorentzVector.Covariant

[HepLean/SpaceTime/LorentzVector/Covariant.lean](#) on GitHub.

def CovariantLorentzVector[\[source\]](#)

The type of covariant Lorentz Vectors in d-space dimensions.

```
def CovariantLorentzVector : Type
```

instance AddCommMonoid (CovariantLorentzVector d)[\[source\]](#)

An instance of an additive commutative monoid on LorentzVector.

```
instance : AddCommMonoid (CovariantLorentzVector d)
```

```
instance Module  $\mathbb{R}$  (CovariantLorentzVector d)
```

[\[source\]](#)

An instance of a module on LorentzVector.

```
noncomputable instance : Module  $\mathbb{R}$  (CovariantLorentzVector d)
```

```
instance AddCommGroup (CovariantLorentzVector d)
```

[\[source\]](#)

```
instance : AddCommGroup (CovariantLorentzVector d)
```

```
instance TopologicalSpace (CovariantLorentzVector d)
```

[\[source\]](#)

The structure of a topological space LorentzVector d.

```
instance : TopologicalSpace (CovariantLorentzVector d)
```

```
def stdBasis
```

[\[source\]](#)

The standard basis of LorentzVector indexed by $\text{Fin } 1 \oplus \text{Fin } (d)$.

```
@[simps!]
```

```
noncomputable def stdBasis : Basis (Fin 1  $\oplus$  Fin (d))  $\mathbb{R}$  (CovariantLorentzVector d)
```

```
lemma decomp_stdBasis
```

[\[source\]](#)

```
lemma decomp_stdBasis (v : CovariantLorentzVector d) :  $\sum i, v\ i \cdot \text{stdBasis } i = v$ 
```

```
lemma decomp_stdBasis'
```

[\[source\]](#)

```
@[simp]
```

```
lemma decomp_stdBasis' (v : CovariantLorentzVector d) :  
  v (Sum.inl 0)  $\cdot$  stdBasis (Sum.inl 0)  
  +  $\sum a_2 : \text{Fin } d, v$  (Sum.inr a2)  $\cdot$  stdBasis (Sum.inr a2) = v
```

```
def rep
```

[\[source\]](#)

The representation of the Lorentz group acting on covariant Lorentz vectors.

```
def rep : Representation  $\mathbb{R}$  (LorentzGroup d) (CovariantLorentzVector d) where
```

```
lemma rep_apply
```

[\[source\]](#)

```
lemma rep_apply (g : LorentzGroup d) : rep g v = (g.1-1)T *v v
```

```
lemma rep_apply_stdBasis
```

[\[source\]](#)

```
lemma rep_apply_stdBasis (g : LorentzGroup d) ( $\mu : \text{Fin } 1 \oplus \text{Fin } d$ ) :  
  rep g (stdBasis  $\mu$ ) =  $\sum v, g^{-1}.1\ \mu\ v \cdot \text{stdBasis } v$ 
```

2.20 HepLean.SpaceTime.LorentzVector.LorentzAction

[HepLean/SpaceTime/LorentzVector/LorentzAction.lean](#) on GitHub.

def rep [source]

The contravariant action of the Lorentz group on a Lorentz vector.

```
def rep : Representation ℝ (LorentzGroup d) (LorentzVector d) where
```

lemma rep_apply [source]

```
lemma rep_apply (g : LorentzGroup d) : rep g v = g *v v
```

lemma rep_apply_stdBasis [source]

```
lemma rep_apply_stdBasis (g : LorentzGroup d) (μ : Fin 1 ⊕ Fin d) :  
  rep g (stdBasis μ) = ∑ v, g.1.transpose μ v • stdBasis v
```

2.21 HepLean.SpaceTime.LorentzVector.NormOne

[HepLean/SpaceTime/LorentzVector/NormOne.lean](#) on GitHub.

def NormOneLorentzVector [source]

The set of Lorentz vectors with norm 1.

```
@[simp]  
def NormOneLorentzVector (d : ℕ) : Set (LorentzVector d)
```

instance TopologicalSpace (NormOneLorentzVector d) [source]

```
instance : TopologicalSpace (NormOneLorentzVector d)
```

lemma NormOneLorentzVector.mem_iff [source]

```
lemma mem_iff {x : LorentzVector d} : x ∈ NormOneLorentzVector d ↔ ⟨⟨x, x⟩⟩m = 1
```

def neg [source]

The negative of a NormOneLorentzVector as a NormOneLorentzVector.

```
def neg : NormOneLorentzVector d
```

lemma time_sq [source]

```
lemma time_sq : v.1.time ^ 2 = 1 + ||v.1.space|| ^ 2
```

lemma abs_time_ge_one [source]

```
lemma abs_time_ge_one : 1 ≤ |v.1.time|
```

lemma norm_space_le_abs_time[\[source\]](#)

```
lemma norm_space_le_abs_time : ||v.1.space|| < |v.1.time|
```

lemma norm_space_leq_abs_time[\[source\]](#)

```
lemma norm_space_leq_abs_time : ||v.1.space|| ≤ |v.1.time|
```

lemma time_le_minus_one_or_ge_one[\[source\]](#)

```
lemma time_le_minus_one_or_ge_one : v.1.time ≤ -1 ∨ 1 ≤ v.1.time
```

lemma time_nonpos_iff[\[source\]](#)

```
lemma time_nonpos_iff : v.1.time ≤ 0 ↔ v.1.time ≤ - 1
```

lemma time_nonneg_iff[\[source\]](#)

```
lemma time_nonneg_iff : 0 ≤ v.1.time ↔ 1 ≤ v.1.time
```

lemma time_pos_iff[\[source\]](#)

```
lemma time_pos_iff : 0 < v.1.time ↔ 1 ≤ v.1.time
```

lemma time_abs_sub_space_norm[\[source\]](#)

```
lemma time_abs_sub_space_norm :  
  0 ≤ |v.1.time| * |w.1.time| - ||v.1.space|| * ||w.1.space||
```

def FuturePointing[\[source\]](#)

The future pointing Lorentz vectors with Norm one.

```
def FuturePointing (d : ℕ) : Set (NormOneLorentzVector d)
```

instance TopologicalSpace (FuturePointing d)[\[source\]](#)

```
instance : TopologicalSpace (FuturePointing d)
```

lemma FuturePointing.mem_iff[\[source\]](#)

```
lemma mem_iff : v ∈ FuturePointing d ↔ 0 < v.1.time
```

lemma mem_iff_time_nonneg[\[source\]](#)

```
lemma mem_iff_time_nonneg : v ∈ FuturePointing d ↔ 0 ≤ v.1.time
```

lemma not_mem_iff[\[source\]](#)

```
lemma not_mem_iff : v ∉ FuturePointing d ↔ v.1.time ≤ 0
```

lemma not_mem_iff_neg[\[source\]](#)

```
lemma not_mem_iff_neg : v ∉ FuturePointing d ↔ neg v ∈ FuturePointing d
```

lemma time_nonneg[\[source\]](#)

```
lemma time_nonneg : 0 ≤ f.1.1.time
```

lemma abs_time[\[source\]](#)

```
lemma abs_time : |f.1.1.time| = f.1.1.time
```

lemma time_eq_sqrt[\[source\]](#)

```
lemma time_eq_sqrt : f.1.1.time = √(1 + ||f.1.1.space|| ^ 2)
```

lemma metric_nonneg[\[source\]](#)

```
lemma metric_nonneg : 0 ≤ ⟨⟨f, f'.1.1⟩⟩m
```

lemma one_add_metric_non_zero[\[source\]](#)

```
lemma one_add_metric_non_zero : 1 + ⟨⟨f, f'.1.1⟩⟩m ≠ 0
```

lemma metric_reflect_mem_mem[\[source\]](#)

```
lemma metric_reflect_mem_mem (h : v ∈ FuturePointing d) (hw : w ∈ FuturePointing d) :  
  0 ≤ ⟨⟨v.1, w.1.spaceReflection⟩⟩m
```

lemma metric_reflect_not_mem_not_mem[\[source\]](#)

```
lemma metric_reflect_not_mem_not_mem (h : v ∉ FuturePointing d) (hw : w ∉ FuturePointing d) :  
  0 ≤ ⟨⟨v.1, w.1.spaceReflection⟩⟩m
```

lemma metric_reflect_mem_not_mem[\[source\]](#)

```
lemma metric_reflect_mem_not_mem (h : v ∈ FuturePointing d) (hw : w ∉ FuturePointing d) :  
  ⟨⟨v.1, w.1.spaceReflection⟩⟩m ≤ 0
```

lemma metric_reflect_not_mem_mem[\[source\]](#)

```
lemma metric_reflect_not_mem_mem (h : v ∉ FuturePointing d) (hw : w ∈ FuturePointing d) :  
  ⟨⟨v.1, w.1.spaceReflection⟩⟩m ≤ 0
```

def timeVecNormOneFuture[\[source\]](#)

The FuturePointing d which has all space components zero.

```
@[simps!]  
noncomputable def timeVecNormOneFuture : FuturePointing d
```


def pathFromTime[\[source\]](#)

A continuous path from timeVecNormOneFuture to any other.

```
noncomputable def pathFromTime (u : FuturePointing d) : Path timeVecNormOneFuture u where
```

lemma isPathConnected[\[source\]](#)

```
lemma isPathConnected : IsPathConnected (@Set.univ (FuturePointing d))
```

lemma metric_continuous[\[source\]](#)

```
lemma metric_continuous (u : LorentzVector d) :  
  Continuous (fun (a : FuturePointing d) => ⟨⟨u, a.1.1⟩⟩_m)
```

2.22 HepLean.SpaceTime.MinkowskiMetric

[HepLean/SpaceTime/MinkowskiMetric.lean](#) on GitHub.

def minkowskiMatrix[\[source\]](#)

The d .succ-dimensional real matrix of the form $\text{diag}(1, -1, -1, -1, \dots)$.

```
def minkowskiMatrix {d : ℕ} : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ
```

lemma sq[\[source\]](#)

```
@[simp]  
lemma sq : @minkowskiMatrix d * minkowskiMatrix = 1
```

lemma eq_transpose[\[source\]](#)

```
@[simp]  
lemma eq_transpose : minkowskiMatrixT = @minkowskiMatrix d
```

lemma det_eq_neg_one_pow_d[\[source\]](#)

```
@[simp]  
lemma det_eq_neg_one_pow_d : (@minkowskiMatrix d).det = (- 1) ^ d
```

lemma η_apply_mul_η_apply_diag[\[source\]](#)

```
@[simp]  
lemma η_apply_mul_η_apply_diag (μ : Fin 1 ⊕ Fin d) : η μ μ * η μ μ = 1
```

lemma as_block[\[source\]](#)

```
lemma as_block : @minkowskiMatrix d = (  
  Matrix.fromBlocks (1 : Matrix (Fin 1) (Fin 1) ℝ) 0 0 (-1 : Matrix (Fin d) (Fin d) ℝ))
```

lemma off_diag_zero[\[source\]](#)`@[simp]``lemma off_diag_zero {μ v : Fin 1 ⊕ Fin d} (h : μ ≠ v) : η μ v = 0`**def minkowskiLinearForm**[\[source\]](#)*Given a Lorentz vector  $v$  we define the linear map  $w \mapsto v * \eta * w$ .*`@[simp!]``def minkowskiLinearForm {d : ℕ} (v : LorentzVector d) : LorentzVector d →ℓ[ℝ] ℝ where`**def minkowskiMetric**[\[source\]](#)*The Minkowski metric as a bilinear map.*`def minkowskiMetric {d : ℕ} : LorentzVector d →ℓ[ℝ] LorentzVector d →ℓ[ℝ] ℝ where`**lemma as\_sum**[\[source\]](#)*The Minkowski metric expressed as a sum.*`lemma as_sum :`
$$\langle\langle v, w \rangle\rangle_m = v.time * w.time - \sum i, v.space\ i * w.space\ i$$
lemma as_sum_self[\[source\]](#)*The Minkowski metric expressed as a sum for a single vector.*`lemma as_sum_self :  $\langle\langle v, v \rangle\rangle_m = v.time^2 - \|v.space\|^2$` **lemma eq_time_minus_inner_prod**[\[source\]](#)`lemma eq_time_minus_inner_prod :  $\langle\langle v, w \rangle\rangle_m = v.time * w.time - \langle\langle v.space, w.space \rangle\rangle_{\mathbb{R}}$` **lemma self_eq_time_minus_norm**[\[source\]](#)`lemma self_eq_time_minus_norm :  $\langle\langle v, v \rangle\rangle_m = v.time^2 - \|v.space\|^2$` **lemma symm**[\[source\]](#)*The Minkowski metric is symmetric in its arguments..*`lemma symm :  $\langle\langle v, w \rangle\rangle_m = \langle\langle w, v \rangle\rangle_m$` **lemma time_sq_eq_metric_add_space**[\[source\]](#)`lemma time_sq_eq_metric_add_space :  $v.time^2 = \langle\langle v, v \rangle\rangle_m + \|v.space\|^2$` **lemma right_spaceReflection**[\[source\]](#)`lemma right_spaceReflection :  $\langle\langle v, w.spaceReflection \rangle\rangle_m = v.time * w.time + \langle\langle v.space, w.space \rangle\rangle_{\mathbb{R}}$`

lemma self_spaceReflection_eq_zero_iff[\[source\]](#)

```
lemma self_spaceReflection_eq_zero_iff :  $\langle\langle v, v.\text{spaceReflection} \rangle\rangle_m = 0 \leftrightarrow v = 0$ 
```

lemma nondegenerate[\[source\]](#)

The metric tensor is non-degenerate.

```
lemma nondegenerate :  $(\forall w, \langle\langle w, v \rangle\rangle_m = 0) \leftrightarrow v = 0$ 
```

lemma leq_time_sq[\[source\]](#)

```
lemma leq_time_sq :  $\langle\langle v, v \rangle\rangle_m \leq v.\text{time}^2$ 
```

lemma ge_abs_inner_product[\[source\]](#)

```
lemma ge_abs_inner_product :  $v.\text{time} * w.\text{time} - \|\langle\langle v.\text{space}, w.\text{space} \rangle\rangle_{\mathbb{R}}\| \leq \langle\langle v, w \rangle\rangle_m$ 
```

lemma ge_sub_norm[\[source\]](#)

```
lemma ge_sub_norm :  $v.\text{time} * w.\text{time} - \|v.\text{space}\| * \|w.\text{space}\| \leq \langle\langle v, w \rangle\rangle_m$ 
```

def dual[\[source\]](#)

The dual of a matrix with respect to the Minkowski metric.

```
def dual : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ
```

lemma dual_id[\[source\]](#)

```
@[simp]
lemma dual_id : @dual d 1 = 1
```

lemma dual_mul[\[source\]](#)

```
@[simp]
lemma dual_mul : dual (Λ * Λ') = dual Λ' * dual Λ
```

lemma dual_dual[\[source\]](#)

```
@[simp]
lemma dual_dual : dual (dual Λ) = Λ
```

lemma dual_eta[\[source\]](#)

```
@[simp]
lemma dual_eta : @dual d η = η
```

lemma dual_transpose[\[source\]](#)

@[simp]

lemma dual_transpose : $\text{dual } \Lambda^T = (\text{dual } \Lambda)^T$ **lemma det_dual**[\[source\]](#)

@[simp]

lemma det_dual : $(\text{dual } \Lambda).\text{det} = \Lambda.\text{det}$ **lemma dual_mulVec_right**[\[source\]](#)

@[simp]

lemma dual_mulVec_right : $\langle\langle x, (\text{dual } \Lambda) *_{\nu} y \rangle\rangle_{\mathfrak{m}} = \langle\langle \Lambda *_{\nu} x, y \rangle\rangle_{\mathfrak{m}}$ **lemma dual_mulVec_left**[\[source\]](#)

@[simp]

lemma dual_mulVec_left : $\langle\langle (\text{dual } \Lambda) *_{\nu} x, y \rangle\rangle_{\mathfrak{m}} = \langle\langle x, \Lambda *_{\nu} y \rangle\rangle_{\mathfrak{m}}$ **lemma matrix_apply_eq_iff_sub**[\[source\]](#)**lemma** matrix_apply_eq_iff_sub : $\langle\langle v, \Lambda *_{\nu} w \rangle\rangle_{\mathfrak{m}} = \langle\langle v, \Lambda' *_{\nu} w \rangle\rangle_{\mathfrak{m}} \leftrightarrow \langle\langle v, (\Lambda - \Lambda') *_{\nu} w \rangle\rangle_{\mathfrak{m}} = 0$ **lemma matrix_eq_iff_eq_forall'**[\[source\]](#)**lemma** matrix_eq_iff_eq_forall' : $(\forall v, \Lambda *_{\nu} v = \Lambda' *_{\nu} v) \leftrightarrow \forall w, v, \langle\langle v, \Lambda *_{\nu} w \rangle\rangle_{\mathfrak{m}} = \langle\langle v, \Lambda' *_{\nu} w \rangle\rangle_{\mathfrak{m}}$ **lemma matrix_eq_iff_eq_forall**[\[source\]](#)**lemma** matrix_eq_iff_eq_forall : $\Lambda = \Lambda' \leftrightarrow \forall w, v, \langle\langle v, \Lambda *_{\nu} w \rangle\rangle_{\mathfrak{m}} = \langle\langle v, \Lambda' *_{\nu} w \rangle\rangle_{\mathfrak{m}}$ **lemma matrix_eq_id_iff**[\[source\]](#)**lemma** matrix_eq_id_iff : $\Lambda = 1 \leftrightarrow \forall w, v, \langle\langle v, \Lambda *_{\nu} w \rangle\rangle_{\mathfrak{m}} = \langle\langle v, w \rangle\rangle_{\mathfrak{m}}$ **lemma basis_left**[\[source\]](#)

@[simp]

lemma basis_left $(\mu : \text{Fin } 1 \oplus \text{Fin } d) : \langle\langle e \mu, v \rangle\rangle_{\mathfrak{m}} = \eta \mu \mu * v \mu$ **lemma on_timeVec**[\[source\]](#)**lemma** on_timeVec : $\langle\langle \text{timeVec}, @\text{timeVec } d \rangle\rangle_{\mathfrak{m}} = 1$ **lemma on_basis_mulVec**[\[source\]](#)**lemma** on_basis_mulVec $(\mu \nu : \text{Fin } 1 \oplus \text{Fin } d) : \langle\langle e \mu, \Lambda *_{\nu} e \nu \rangle\rangle_{\mathfrak{m}} = \eta \mu \mu * \Lambda \mu \nu$

lemma on_basis[\[source\]](#)

```
lemma on_basis (μ v : Fin 1 ⊕ Fin d) : ⟨⟨e μ, e v⟩⟩m = η μ v
```

lemma matrix_apply_stdBasis[\[source\]](#)

```
lemma matrix_apply_stdBasis (v μ : Fin 1 ⊕ Fin d) :  
  Λ v μ = η v v * ⟨⟨e v, Λ *v e μ⟩⟩m
```

2.23 HepLean.SpaceTime.SL2C.Basic

[HepLean/SpaceTime/SL2C/Basic.lean](#) on GitHub.

def repLorentzVector[\[source\]](#)

The representation of $SL(2, \mathbb{C})$ on $spaceTime$ obtained from $toSelfAdjointMatrix$ and $repSelfAdjointMatrix$.

```
def repLorentzVector : Representation ℝ SL(2, ℂ) (LorentzVector 3) where
```

lemma iff_det_selfAdjoint[\[source\]](#)

```
lemma iff_det_selfAdjoint (Λ : Matrix (Fin 1 ⊕ Fin 3) (Fin 1 ⊕ Fin 3) ℝ) : Λ ∈ LorentzGroup 3 ↔  
  ∀ (x : selfAdjoint (Matrix (Fin 2) (Fin 2) ℂ)),  
  det ((toSelfAdjointMatrix ◦  
    toLin LorentzVector.stdBasis LorentzVector.stdBasis Λ ◦ toSelfAdjointMatrix.symm) x).1  
  = det x.1
```

2.24 HepLean.StandardModel.HiggsBoson.Basic

[HepLean/StandardModel/HiggsBoson/Basic.lean](#) on GitHub.

def toFin2ℂ[\[source\]](#)

The continuous linear map from the vector space $HiggsVec$ to $(Fin\ 2 \rightarrow \mathbb{C})$ achieved by casting vectors.

```
def toFin2ℂ : HiggsVec →L[ℝ] (Fin 2 → ℂ) where
```

lemma smooth_toFin2ℂ[\[source\]](#)

The map $toFin2\mathbb{C}$ is smooth.

```
lemma smooth_toFin2ℂ : Smooth ℐ(ℝ, HiggsVec) ℐ(ℝ, Fin 2 → ℂ) toFin2ℂ
```

lemma toFin2ℂ_comp_toHiggsVec[\[source\]](#)

```
lemma toFin2ℂ_comp_toHiggsVec (φ : HiggsField) :  
  toFin2ℂ ◦ φ.toHiggsVec = φ
```

2.25 HepLean.StandardModel.HiggsBoson.PointwiseInnerProd

[HepLean/StandardModel/HiggsBoson/PointwiseInnerProd.lean](#) on GitHub.

lemma innerProd_left_zero[\[source\]](#)

```
@[simp]
lemma innerProd_left_zero (φ : HiggsField) : ⟨⟨0, φ⟩⟩_H = 0
```

lemma innerProd_right_zero[\[source\]](#)

```
@[simp]
lemma innerProd_right_zero (φ : HiggsField) : ⟨⟨φ, 0⟩⟩_H = 0
```

lemma innerProd_expand[\[source\]](#)

```
lemma innerProd_expand (φ1 φ2 : HiggsField) :
  ⟨⟨φ1, φ2⟩⟩_H = fun x => equivRealProdCLM.symm (((φ1 x 0).re * (φ2 x 0).re
+ (φ1 x 1).re * (φ2 x 1).re + (φ1 x 0).im * (φ2 x 0).im + (φ1 x 1).im * (φ2 x 1).im),
  ((φ1 x 0).re * (φ2 x 0).im + (φ1 x 1).re * (φ2 x 1).im
  - (φ1 x 0).im * (φ2 x 0).re - (φ1 x 1).im * (φ2 x 1).re))
```

lemma smooth_innerProd[\[source\]](#)

```
lemma smooth_innerProd (φ1 φ2 : HiggsField) : Smooth  $\mathcal{I}(\mathbb{R}, \text{SpaceTime}) \mathcal{I}(\mathbb{R}, \mathbb{C})$  ⟨⟨φ1, φ2⟩⟩_H
```

lemma innerProd_self_eq_normSq[\[source\]](#)

```
lemma innerProd_self_eq_normSq (φ : HiggsField) (x : SpaceTime) :
  ⟨⟨φ, φ⟩⟩_H x = ||φ||_H ^ 2 x
```

lemma normSq_eq_innerProd_self[\[source\]](#)

```
lemma normSq_eq_innerProd_self (φ : HiggsField) (x : SpaceTime) :
  ||φ x|| ^ 2 = (⟨⟨φ, φ⟩⟩_H x).re
```

2.26 HepLean.StandardModel.HiggsBoson.Potential

[HepLean/StandardModel/HiggsBoson/Potential.lean](#) on GitHub.

lemma complete_square[\[source\]](#)

```
lemma complete_square (μ2 ↓ : ℝ) (h : ↓ ≠ 0) (φ : HiggsField) (x : SpaceTime) :
  potential μ2 ↓ φ x = ↓ * (||φ||_H ^ 2 x - μ2 / (2 * ↓)) ^ 2 - μ2 ^ 2 / (4 * ↓)
```

def IsBounded[\[source\]](#)

The proposition on the coefficents for a potential to be bounded.

```
def IsBounded (μ2 ↓ : ℝ) : Prop
```

lemma snd_term_nonneg[\[source\]](#)

The second term of the potential is non-negative.

```
lemma snd_term_nonneg (φ : HiggsField) (x : SpaceTime) :
  0 ≤ ↓ * ||φ||_H ^ 2 x * ||φ||_H ^ 2 x
```

lemma as_quad[\[source\]](#)

```
lemma as_quad (μ2 ↑ : ℝ) (φ : HiggsField) (x : SpaceTime) :
  ↑ * ||φ||_H ^ 2 x * ||φ||_H ^ 2 x + (- μ2) * ||φ||_H ^ 2 x + (- potential μ2 ↑ φ x) = 0
```

lemma discrim_nonneg[\[source\]](#)

The discriminant of the quadratic formed by the potential is non-negative.

```
lemma discrim_nonneg (φ : HiggsField) (x : SpaceTime) :
  0 ≤ discrim ↑ (- μ2) (- potential μ2 ↑ φ x)
```

lemma eq_zero_at[\[source\]](#)

```
lemma eq_zero_at (φ : HiggsField) (x : SpaceTime)
  (hV : potential μ2 ↑ φ x = 0) : φ x = 0 ∨ ||φ||_H ^ 2 x = μ2 / ↑
```

lemma eq_zero_at_of_μSq_nonpos[\[source\]](#)

```
lemma eq_zero_at_of_μSq_nonpos {μ2 : ℝ} (hμ2 : μ2 ≤ 0)
  (φ : HiggsField) (x : SpaceTime) (hV : potential μ2 ↑ φ x = 0) : φ x = 0
```

lemma bounded_below[\[source\]](#)

```
lemma bounded_below (φ : HiggsField) (x : SpaceTime) :
  - μ2 ^ 2 / (4 * ↑) ≤ potential μ2 ↑ φ x
```

lemma bounded_below_of_μSq_nonpos[\[source\]](#)

```
lemma bounded_below_of_μSq_nonpos {μ2 : ℝ}
  (hμSq : μ2 ≤ 0) (φ : HiggsField) (x : SpaceTime) : 0 ≤ potential μ2 ↑ φ x
```

lemma discrim_eq_zero_of_eq_bound[\[source\]](#)

```
lemma discrim_eq_zero_of_eq_bound (φ : HiggsField) (x : SpaceTime)
  (hV : potential μ2 ↑ φ x = - μ2 ^ 2 / (4 * ↑)) :
  discrim ↑ (- μ2) (- potential μ2 ↑ φ x) = 0
```

lemma normSq_of_eq_bound[\[source\]](#)

```
lemma normSq_of_eq_bound (φ : HiggsField) (x : SpaceTime)
  (hV : potential μ2 ↑ φ x = - μ2 ^ 2 / (4 * ↑)) :
  ||φ||_H ^ 2 x = μ2 / (2 * ↑)
```

lemma eq_bound_iff[\[source\]](#)

```
lemma eq_bound_iff (φ : HiggsField) (x : SpaceTime) :
  potential μ2 ↑ φ x = - μ2 ^ 2 / (4 * ↑) ↔ ||φ||_H ^ 2 x = μ2 / (2 * ↑)
```

lemma eq_bound_iff_of_μSq_nonpos[\[source\]](#)

```
lemma eq_bound_iff_of_μSq_nonpos {μ2 : ℝ}
  (hμ2 : μ2 ≤ 0) (φ : HiggsField) (x : SpaceTime) :
  potential μ2 ⋈ φ x = 0 ↔ φ x = 0
```

lemma eq_bound_IsMin0n[\[source\]](#)

```
lemma eq_bound_IsMin0n (φ : HiggsField) (x : SpaceTime)
  (hv : potential μ2 ⋈ φ x = - μ2 ^ 2 / (4 * ⋈)) :
  IsMin0n (fun (φ, x) => potential μ2 ⋈ φ x) Set.univ (φ, x)
```

lemma eq_bound_IsMin0n_of_μSq_nonpos[\[source\]](#)

```
lemma eq_bound_IsMin0n_of_μSq_nonpos {μ2 : ℝ}
  (hμ2 : μ2 ≤ 0) (φ : HiggsField) (x : SpaceTime) (hv : potential μ2 ⋈ φ x = 0) :
  IsMin0n (fun (φ, x) => potential μ2 ⋈ φ x) Set.univ (φ, x)
```

lemma bound_reached_of_μSq_nonneg[\[source\]](#)

```
lemma bound_reached_of_μSq_nonneg {μ2 : ℝ} (hμ2 : 0 ≤ μ2) :
  ∃ (φ : HiggsField) (x : SpaceTime),
  potential μ2 ⋈ φ x = - μ2 ^ 2 / (4 * ⋈)
```

lemma IsMin0n_iff_of_μSq_nonneg[\[source\]](#)

```
lemma IsMin0n_iff_of_μSq_nonneg {μ2 : ℝ} (hμ2 : 0 ≤ μ2) :
  IsMin0n (fun (φ, x) => potential μ2 ⋈ φ x) Set.univ (φ, x) ↔
  ||φ||_H ^ 2 x = μ2 / (2 * ⋈)
```

lemma IsMin0n_iff_of_μSq_nonpos[\[source\]](#)

```
lemma IsMin0n_iff_of_μSq_nonpos {μ2 : ℝ} (hμ2 : μ2 ≤ 0) :
  IsMin0n (fun (φ, x) => potential μ2 ⋈ φ x) Set.univ (φ, x) ↔ φ x = 0
```

2.27 scripts.find_TODOs

[scripts/find_TODOs.lean](#) on GitHub.

def HepLeanTODOItem[\[source\]](#)

Given a list of lines, outputs an error message and a line number.

```
def HepLeanTODOItem : Type
```

def TODOFinder[\[source\]](#)

Checks if a .

```
def TODOFinder : HepLeanTODOItem
```


structure TODOContext[\[source\]](#)

```

structure TODOContext where
  statement : String
  lineNumber : ℕ
  path : FilePath

```

def printTODO[\[source\]](#)

```

def printTODO (todos : Array TODOContext) : IO Unit

```

def filePathToGitPath[\[source\]](#)

```

def filePathToGitPath (S : FilePath) (n : ℕ) : String

```

def docTODO[\[source\]](#)

```

def docTODO (todos : Array TODOContext) : IO String

```

def hepLeanLintFile[\[source\]](#)

```

def hepLeanLintFile (path : FilePath) : IO String

```

def todoFileHeader[\[source\]](#)

```

def todoFileHeader : String

```

2.28 scripts.hepLean_style_lint

[scripts/hepLean_style_lint.lean](#) on GitHub.

def HepLeanTextLinter[\[source\]](#)

Given a list of lines, outputs an error message and a line number.

```

def HepLeanTextLinter : Type

```

def doubleEmptyLineLinter[\[source\]](#)

Checks if there are two consecutive empty lines.

```

def doubleEmptyLineLinter : HepLeanTextLinter

```

def doubleSpaceLinter[\[source\]](#)

Checks if there is a double space in the line, which is not at the start.

```

def doubleSpaceLinter : HepLeanTextLinter

```

def substringLinter [\[source\]](#)

Substring linter.

```
def substringLinter (s : String) : HepLeanTextLinter
```

def numInitialSpacesEven [\[source\]](#)

Number of space at new line must be even.

```
def numInitialSpacesEven : HepLeanTextLinter
```

structure HepLeanErrorContext [\[source\]](#)

```
structure HepLeanErrorContext where
  error : String
  lineNumber : ℕ
  columnNumber : ℕ
  path : FilePath
```

def printErrors [\[source\]](#)

```
def printErrors (errors : Array HepLeanErrorContext) : IO Unit
```

def hepLeanLintFile [\[source\]](#)

```
def hepLeanLintFile (path : FilePath) : IO (Array HepLeanErrorContext)
```

2.29 scripts.mathlib_textLint_on_hepLean

[scripts/mathlib_textLint_on_hepLean.lean](#) on GitHub.

def lintStyleCli [\[source\]](#)

Implementation of the lint_style command line program.

```
def lintStyleCli (args : Cli.Parsed) : IO UInt32
```

def hepLeanLintStyle [\[source\]](#)

Setting up command line options and help text for lake exe lint_style.

```
def hepLeanLintStyle : Cmd
```

2.30 scripts.openAI_doc_check

[scripts/openAI_doc_check.lean](#) on GitHub.

def getDocString [\[source\]](#)

```
def getDocString (constName : Array ConstantInfo) : MetaM String
```

def header[\[source\]](#)

```
def header : String
```

2.31 scripts.stats

[scripts/stats.lean](#) on GitHub.

structure FileStats[\[source\]](#)

```
structure FileStats where
  numDefs : Nat
  numLemsWithoutDocString : Nat
  numLemsWithDocString : Nat
```

def add[\[source\]](#)

```
def FileStats.add (a b : FileStats) : FileStats where
```

def zero[\[source\]](#)

```
def FileStats.zero : FileStats where
```

instance ToString FileStats[\[source\]](#)

```
instance : ToString FileStats where
```

def getConstantDoc[\[source\]](#)

```
def getConstantDoc (constName : Array ConstantInfo) : MetaM (FileStats)
```

def getStats[\[source\]](#)

```
def getStats (imp : Import) : MetaM FileStats
```

def getStatsArray[\[source\]](#)

```
def getStatsArray (a : Array Import) : MetaM (Array FileStats)
```